



MACHINE LEARNING CON PYTHON

TEMAS 3, 4 y 5. Análisis,
Preparación de Datos y
Visualización



Contenidos Temas 4 y 5

- / 1. Introducción y Conceptos Básicos del Análisis de Datos**
- / 2. Librerías Python: NumPy, Pandas, Polars**
- / 3. Visualización de datos: matplotlib, seaborn, plotly**
- / 4. Análisis Exploratorio de Datos (EDA)**

1. INTRODUCCIÓN Y CONCEPTOS BÁSICOS DEL ANÁLISIS DE DATOS

El **análisis de datos** es una **destreza crítica** en un mundo como el actual: un **mundo rodeado y guiado por datos**. Ejemplos:

1. Analista de negocio que intenta obtener información útil y con sentido de datos de ventas.
2. Científico/a responsable de analizar resultados de reiterados experimentos.

Es esencial entender los fundamentos del proceso de análisis de datos, pero ...

¿Qué es el Análisis de Datos?

El análisis de datos es el proceso de **recolectar, inspeccionar, limpiar, transformar, e interpretar datos** para **descubrir información útil, obtener conclusiones**, y brindar apoyo a la **toma de decisiones**.

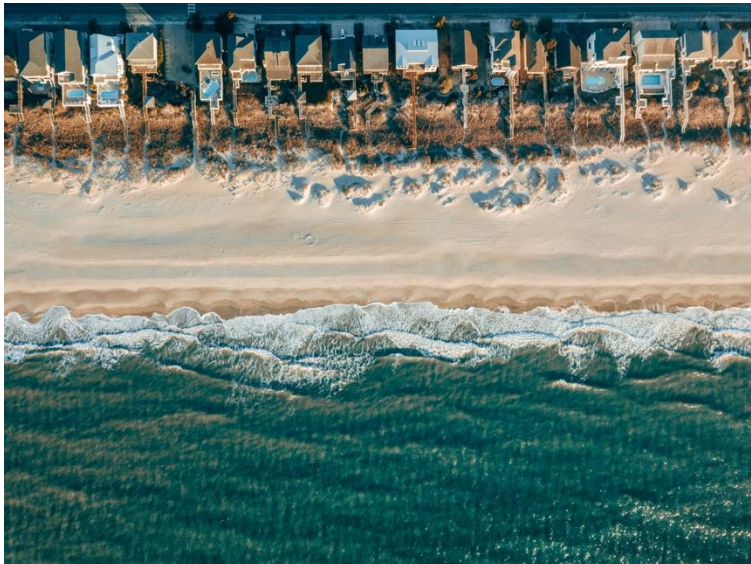
El análisis de datos es el proceso de **recolectar inspeccionar, limpiar, transformar, e interpretar datos** para **descubrir información útil, obtener conclusiones**, y brindar apoyo a la **toma de decisiones**.

- Análisis de Datos = Enfoque sistemático para **entender patrones, tendencias, y relaciones** en un conjunto de datos (*dataset*)



Datos vs. Información vs. Conocimiento

DATOS	Hechos (valores) en crudo, sin contexto ni interpretación	36°C (Tokio, 36°C, 30-07-2025)
INFORMACIÓN	Datos organizados y con contexto	“Hoy la temperatura en Tokio es de 36°C”
CONOCIMIENTO	Comprensión de la información para la toma de decisiones	“Debo beber mucho líquido y quedarme a la sombra si estoy en Tokio hoy”



Transformar datos en conocimiento es el objetivo final del análisis de datos.

1. INTRODUCCIÓN Y CONCEPTOS BÁSICOS DEL ANÁLISIS DE DATOS

¿Qué es un **Dataset**?

- Un Dataset es un **conjunto o colección de datos** recopilados con un propósito específicos en un dominio determinado.
- Pueden estar organizados de forma **estructurada, semi-estructurada, o no estructurada**.

Datos Estructurados	Datos organizados en filas y columnas.	Tabla de clientes con atributos (columnas) como edad, ciudad de residencia, y fecha de última compra
Datos Semi-estructurados	Datos con una estructura flexible pero no estrictamente tabular.	Archivos XML o JSON con detalle de productos
Datos No Estructurados	Datos que no siguen un formato fijo.	Colección de reseñas de texto sobre productos, conjunto de imágenes de productos, audios de canciones.

1. INTRODUCCIÓN Y CONCEPTOS BÁSICOS DEL ANÁLISIS DE DATOS

Fruta	Cantidad	Color
Manzana	4	Rojo
Banana	2	Amarillo
Uva	10	Morado

```
{
  "producto": "Zapatilla",
  "precio": 49.99,
  "tallas": [38, 39, 40]
}
```



“Muy cómodo y fácil de usar. Lo recomiendo.”



“La batería dura poco, pero el diseño es bonito.”

El análisis de datos puede aplicarse a cualquier tipo de dataset, aunque la forma de analizarlos cambiará según su estructura y tipo

Datos Estructurados	Datos organizados en filas y columnas.	Tabla de clientes con atributos (columnas) como edad, ciudad de residencia, y fecha de última compra
Datos Semi-estructurados	Datos con una estructura flexible pero no estrictamente tabular.	Archivos XML o JSON con detalle de productos
Datos No Estructurados	Datos que no siguen un formato fijo.	Colección de reseñas de texto sobre productos, conjunto de imágenes de productos, audios de canciones.

1. INTRODUCCIÓN Y CONCEPTOS BÁSICOS DEL ANÁLISIS DE DATOS

Patrones, tendencias, y relaciones en los datos

- **Patrón:** una característica que se repite de forma frecuente o habitual en un *Dataset*.

→ *Muchos clientes realizan mayores compras los viernes.*

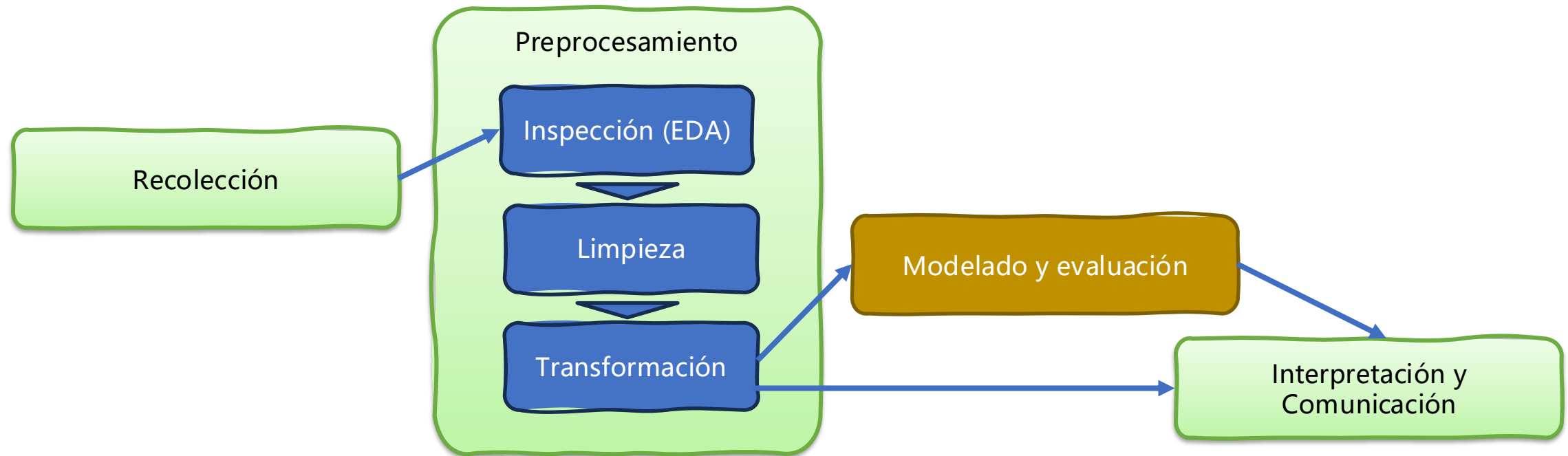
- **Tendencia:** un cambio o evolución en los valores de los datos de forma consistente en el tiempo.

→ *Las ventas online mensuales han ido aumentando en el último semestre.*

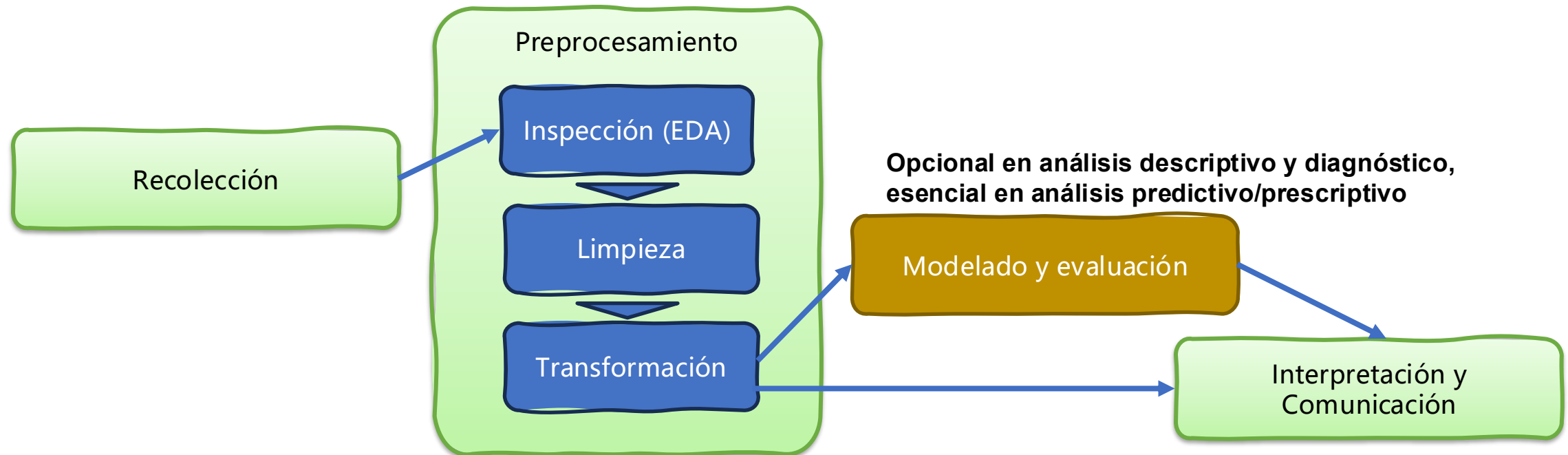
- **Relación:** una conexión entre dos variables de los datos (no necesariamente implicando causalidad).

→ A mayor edad, menor es el tiempo de uso diario de redes sociales

Observar estos fenómenos permite interpretar el "comportamiento" de los datos.



1. INTRODUCCIÓN Y CONCEPTOS BÁSICOS DEL ANÁLISIS DE DATOS



Tipo de análisis

Descriptivo

Diagnóstico

Predictivo

Prescriptivo

Cuestión clave a responder

¿Qué ha ocurrido?

¿Por qué ha ocurrido?

¿Qué pasará?

¿Qué debo hacer?

Cambios en fases clave

Máyor énfasis en inspección, limpieza, resumen de datos pasados/presentes.

Énfasis en visualización, segmentación, comparaciones, tests estadísticos.

Aparece **modelado predictivo (ML)** como fase clave.

Añade **técnicas de optimización y simulación**, además de modelos.

1. INTRODUCCIÓN Y CONCEPTOS BÁSICOS DEL ANÁLISIS DE DATOS

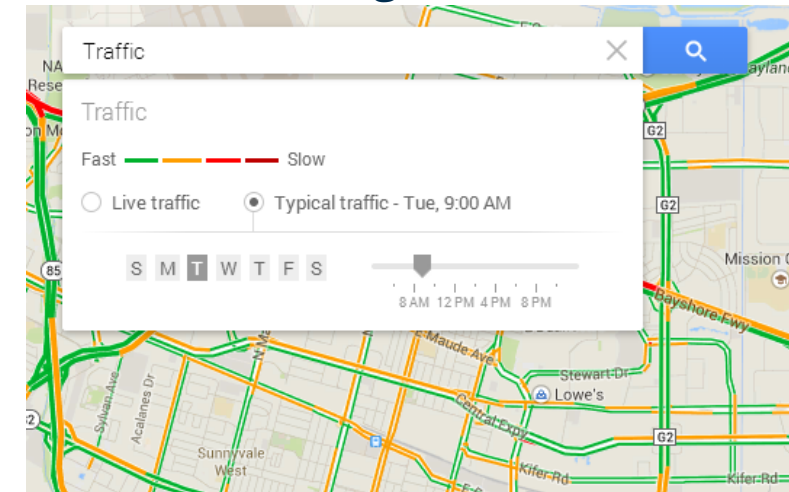
FASE 1. RECOLECCIÓN DE DATOS

En todo análisis de datos, la obtención o recolección de datos es el punto de partida.

- **Recolección:** obtener datos desde diversas fuentes internas o externas, de forma manual o automatizada.

Herramientas y técnicas: APIs (Twitter API, OpenWeather), datos públicos en abierto, Web scrapping, bases de datos (SQL, MongoDB), formularios, sensores IoT, logs, archivos CSV/Excel...

Ejemplo: recogida de datos de movilidad urbana utilizando sensores de tráfico junto con la API de Google Maps.



Sin datos, no hay análisis. Pero no cualquier dato sirve, necesitamos datos relevantes y de calidad.

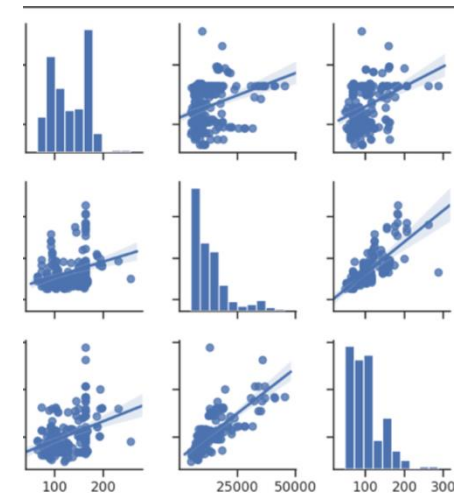
FASE 2. INSPECCIÓN Y EDA (Análisis Exploratorio de Datos)

Conocer los datos que hemos obtenido.

- **Inspección/EDA:** Explorar los datos para tener una comprensión inicial de su estructura, nivel de calidad, y comportamientos generales.

Herramientas y técnicas: Python (Pandas, Matplotlib, Seaborn), R (ggplot2, dplyr), cuadros de mando o dashboards (Power BI, Tableau, Looker), estadística descriptiva.

Ejemplo: detectar a simple vista que los usuarios con más compras pertenecen a una franja de edad específica



No modelos, predigas, o busques conclusiones sin mirar primero.

FASE 3. LIMPIEZA DE DATOS

Depurar para hacer los datos más confiables

- **Limpieza:** eliminación de errores, inconsistencias, y tratamiento de valores faltantes (*missing values*) que pueden distorsionar el posterior análisis.

Herramientas y técnicas: Python (Pandas), OpenRefine, validaciones lógicas según contexto

dropna(), fillna(), duplicated(), ...

Ejemplo: corrección de registros de edad con valores imposibles como 999, edades negativas o 0, etc.



“Garbage in, Garbage out”. Malos datos conducirán a malos resultados.

1. INTRODUCCIÓN Y CONCEPTOS BÁSICOS DEL ANÁLISIS DE DATOS

FASE 4. TRANSFORMACIÓN DE DATOS

Preparar los datos para un análisis más eficaz

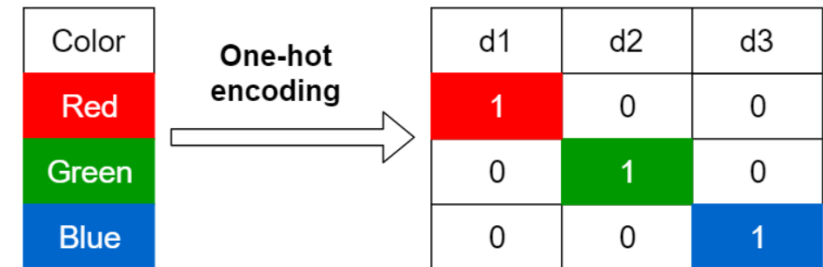
- **Transformación:** reformatear, unificar, combinar, o escalar datos para que puedan ser analizados correctamente.

Herramientas y técnicas: Pandas, feature engineering (ingeniería de características), normalización, codificación de variables discretas y cualitativas, agregaciones, etc.

SQL (GROUP BY, JOIN)

Pandas (groupby(), merge())

Ejemplo: convertir una columna “fecha de compra” en varias variables nuevas como “mes”, “día de la semana”, “fin de semana”, etc.



Transformar los datos pasa por entender lo que el modelo necesita para realizar un buen análisis

FASE 5. MODELADO Y EVALUACIÓN

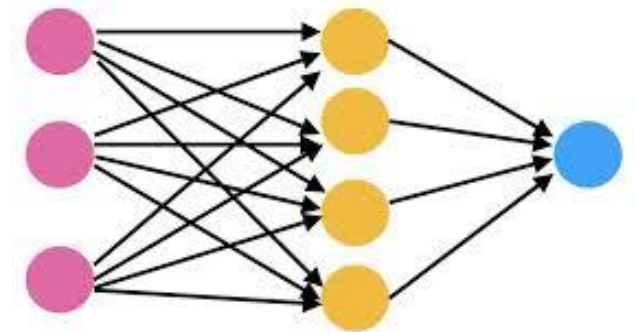
Aprender de los datos

- **Modelado y evaluación:** aplicar algoritmos y construir modelos para descubrir patrones o hacer predicciones, evaluando su rendimiento

Herramientas y técnicas: Python (Scikit-learn, TensorFlow), aprendizaje automático (machine learning), regresión, árboles de decisión, clustering, redes neuronales, etc.

Métricas de evaluación: accuracy, RMSE, AUC, etc., validación cruzada, etc.

Ejemplo: predecir la probabilidad de abandono de clientes (*churn*) con un modelo de clasificación



Un buen modelo no solo predice: generaliza su capacidad predictiva a datos futuros

1. INTRODUCCIÓN Y CONCEPTOS BÁSICOS DEL ANÁLISIS DE DATOS

FASE 6. INTERPRETACIÓN Y COMUNICACIÓN

Contar la historia que nos transmiten los datos (*data storytelling*)

- **Interpretación y comunicación:** extraer conclusiones útiles y accionables (*insights*) y comunicarlas de forma clara a quienes toman decisiones.

Herramientas y técnicas: Python (Matplotlib, Seaborn), software de visualización y Business Intelligence (Tableau, Power BI, Seaborn), lenguaje claro, **storytelling**, métricas clave (KPI), dashboards interactivos.

7 Steps to Master the Art of Data Storytelling:

<https://www.kdnuggets.com/7-steps-to-master-the-art-of-data-storytelling>

Ejemplo: presentar a la dirección de la empresa un *dashboard* que muestra cómo la rotación de empleados afecta a la productividad.



Los datos no cambian nada hasta que alguien los entiende y pasa a la acción.

1. INTRODUCCIÓN Y CONCEPTOS BÁSICOS DEL ANÁLISIS DE DATOS

VISUALIZACIÓN DE DATOS

La **visualización de datos** permite transformar información compleja en representaciones visuales claras, facilitando el análisis, la toma de decisiones y la comunicación de hallazgos de forma efectiva.

Se rige por un conjunto de principios, entre los que podemos destacar:

	¿Por qué importa?	Ejemplo de su aplicación en Python
Claridad	Evita confusión o ruido visual	Usar títulos, etiquetas (<code>xlabel</code> , <code>ylabel</code>), leyendas claras
Simplicidad	Menos es más: no sobrecargar de información	Evitar decoraciones innecesarias o marcadores muy grandes con datos densos.
Coherencia visual	Un estilo uniforme facilita la lectura y comparación	Usar paletas (<code>sns.set_theme()</code> , <code>palette</code>) coherentes
Enfoque en el mensaje	La visualización ha de resaltar lo importante	Elegir el tipo de gráfico adecuado al dato y objetivo

Python, a través de su gran riqueza de librerías de terceros, puede brindar apoyo a lo largo de todo el ciclo de vida o flujo de trabajo del análisis de datos.

¿Por qué Python para análisis de datos?

Por su equilibrio entre simplicidad, potencia, y ecosistema de librerías y *frameworks*.

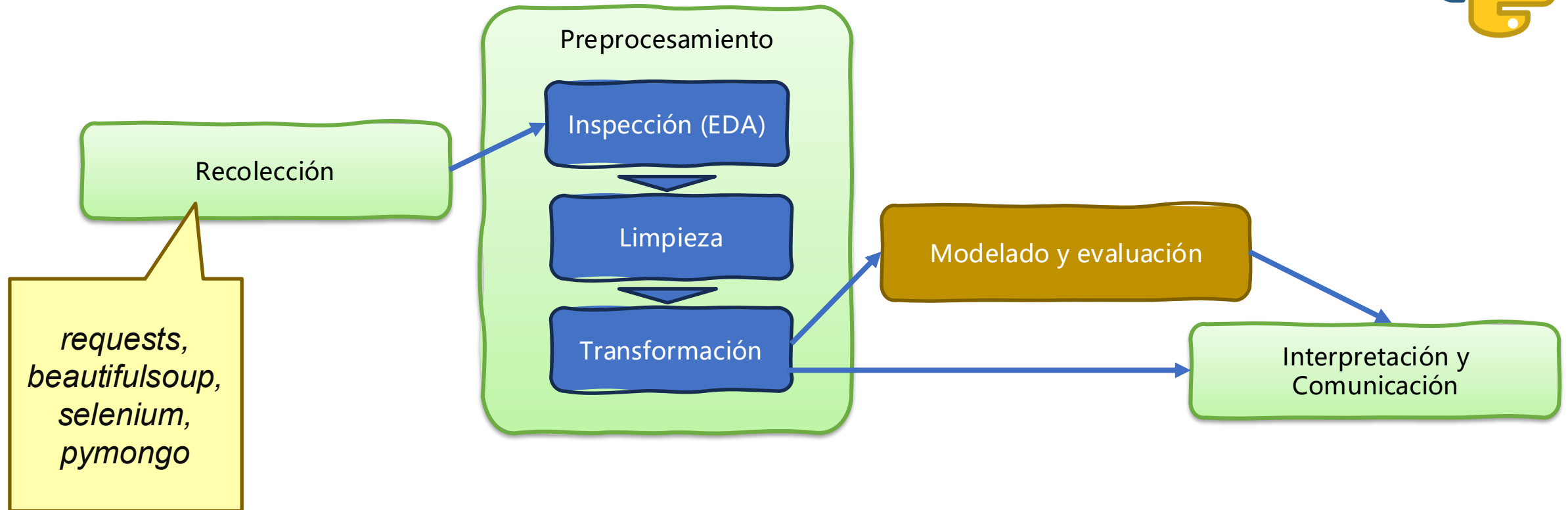
- Sintaxis clara y amigable.
- Gran comunidad y abundante documentación
- Riqueza de librerías.
- Escalable desde simples análisis exploratorios (EDA) en un notebook, hasta flujos de trabajo (*pipelines*) de *machine learning* en producción.

Python, por tanto, **democratiza** el análisis de datos sin sacrificar eficacia.

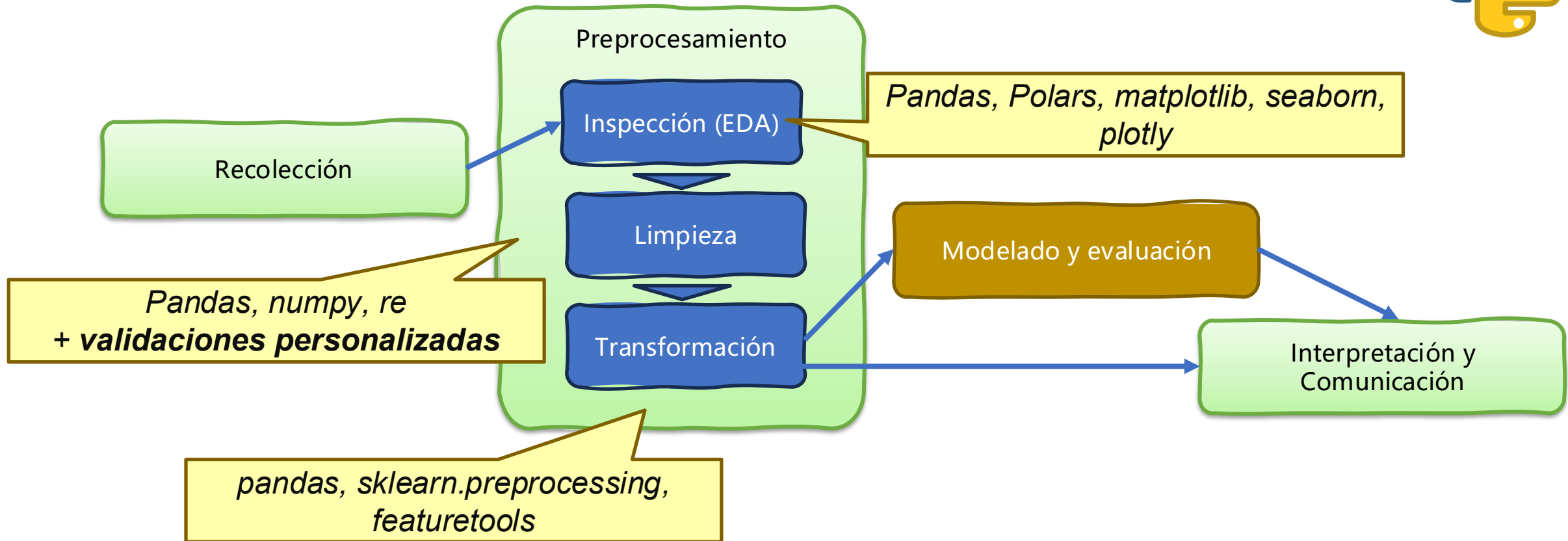


```
import pandas as pd
df = pd.read_csv("ventas.csv")
df.describe()
```

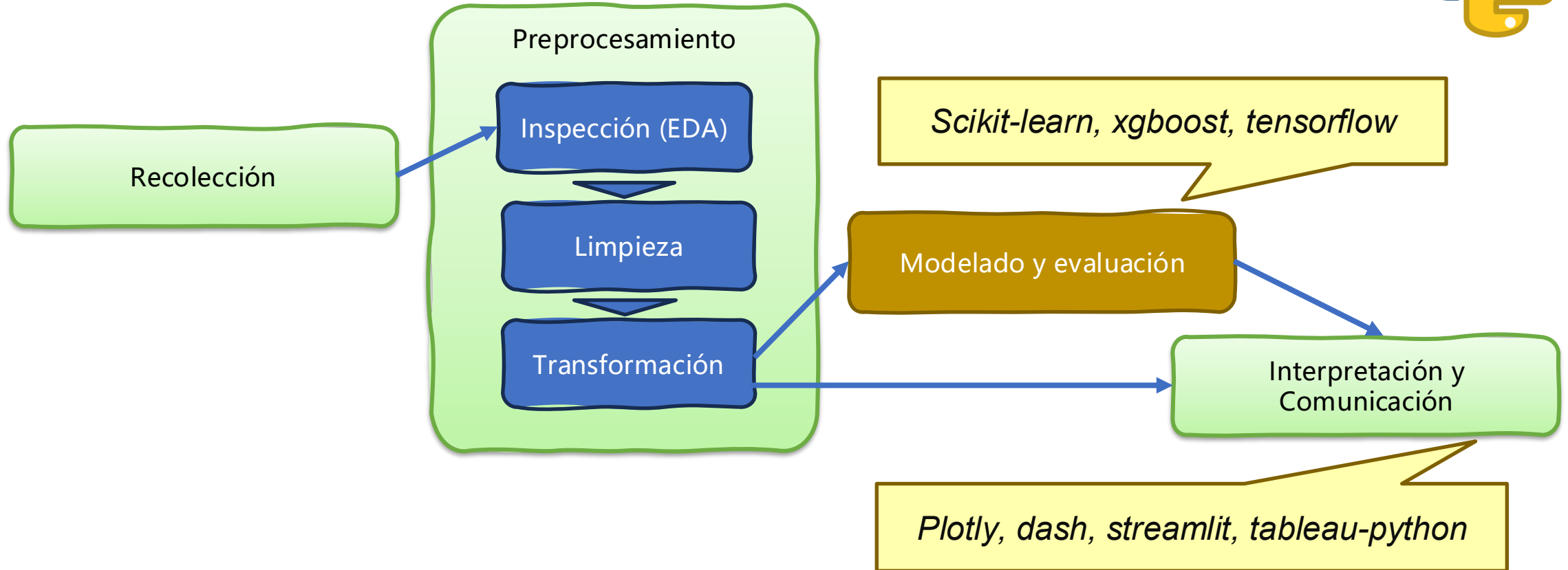
Python a lo largo del flujo de trabajo de análisis de datos, ofrece herramientas de soporte:



Python a lo largo del flujo de trabajo de análisis de datos, ofrece herramientas de soporte:



Python a lo largo del flujo de trabajo de análisis de datos, ofrece herramientas de soporte:



Notebooks interactivos Python (Jupyter/Colab)

Permiten experimentar, documentar y visualizar todo en un mismo entorno.



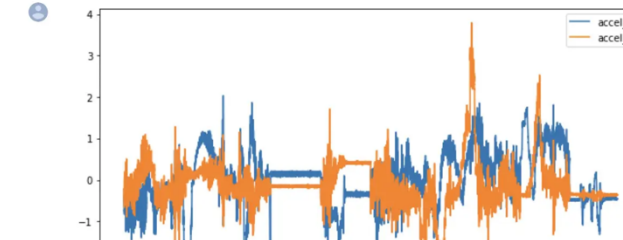
Principales ventajas:

- Código + visualizaciones + texto explicativo (*markdown*) → Todo en un mismo lugar
- Perfecto para exploración de datos, *storytelling*, y análisis colaborativo.
- **Google Colab** permite ejecución en la nube sin instalar nada en tu equipo.
- Código **fácilmente reproducible y compartible**.

We can take advantage of Data Platform's ability to fetch messages across multiple recording sessions by specifying the time range we want in our `get_messages` call.

```
[7] imu_messages = {  
    "time": pd.Timestamp(message.header.stamp.to_nsec(), unit="ns").isoformat(),  
    "accel_x": message.linear_acceleration.x,  
    "accel_y": message.linear_acceleration.y,  
}  
for topic, record, message in client.get_messages(  
    device_id=coverage[0]["device_id"],  
    start=coverage[0]["start"],  
    end=coverage[-1]["end"],  
    topics=["/imu"],  
):  
    imu_messages[topic] = message
```

```
pd.DataFrame(imu_messages).plot(x="time", figsize=(10, 6), rot=45);
```



Los notebooks son, a día de hoy, el laboratorio del analista de datos

BUENAS PRÁCTICAS EN EL ANÁLISIS DE DATOS CON PYTHON

Un buen análisis de datos no es solo un proceso técnico: también es claro, reproducible, y bien documentado.

Lista de recomendaciones:

- Comentar y documentar tu código
- Utilizar nombres de variables relevantes y significativos
- Dividir el código por secciones, por ejemplo, las etapas del flujo.
- Salvaguardar versiones originales de los datos.
- En aplicaciones reales/ de empresa, usar entornos virtuales: *venv*, *conda*. (permite trabajar en proyectos que tienen sus propias dependencias, sin dar lugar a conflictos).
- Visualizar y validar cada paso del proceso de análisis: no esperar al final.
- Antes de compartir un notebook, limpiar celdas innecesarias, salidas de ejecuciones, etc.



2. LIBRERÍAS PYTHON: NUMPY, PANDAS, POLARS

CASOS DE USO REALES DE ANÁLISIS DE DATOS EN PYTHON

Python se utiliza en sectores de lo más diverso para resolver problemas reales con los **datos** como protagonista.

Caso	Descripción	 Herramientas Python útiles
Retail	Agrupar (segmentar) clientes según comportamiento de compra, predecir demanda de productos.	pandas, scikit-learn, matplotlib
Salud	Identificar anomalías en signos vitales o imágenes médicas.	numpy, pandas, xgboost, tensorflow
Finanzas	Calcular riesgos de crédito y prever impagos.	scikit-learn, pandas, matplotlib
Recursos Humanos	Analizar causas de rotación de RRHH y evaluar el clima laboral con encuestas.	seaborn, pandas, wordcloud, matplotlib
Transporte	Optimizar rutas de reparto y analizar tiempos de viaje con datos de GPS.	geopandas, pandas, scikit-learn

NumPy – Álgebra lineal, estadística y cálculo eficientes

NumPy maneja **arrays multidimensionales** (*ndarrays*) de forma más rápida y compacta que las listas nativas de Python → Ideal para **operaciones numéricas vectorizadas**.

```
import numpy as np
a = np.array([1, 2, 3])
print(a * 2)
```



Pandas – organización y manipulación de datos estructurados

- 2 estructuras de datos clave: **Series** (unidimensional), **DataFrame** (tabular, filas-columnas)
- Facilita la lectura, filtrado y preprocesamiento, agrupación, transformación, ...

```
import pandas as pd
df = pd.DataFrame({"Edad": [23, 45], "Ciudad": ["Madrid", "Sevilla"]})
print(df)
```

	Edad	Ciudad
0	23	Madrid
1	45	Sevilla

Numpy ofrece principalmente una estructura de datos multidimensional y eficiente: el **ndarray** (array n-dimensional, o simplemente, array).

Un array puede ser unidimensional (vector), bidimensional (matriz), o de dimensión 3 o superior (tensor).

$$1 \quad \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$\begin{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} & \begin{bmatrix} 3 & 2 \end{bmatrix} \\ \begin{bmatrix} 1 & 7 \end{bmatrix} & \begin{bmatrix} 5 & 4 \end{bmatrix} \end{bmatrix}$$

Escalar: 0-dimensional

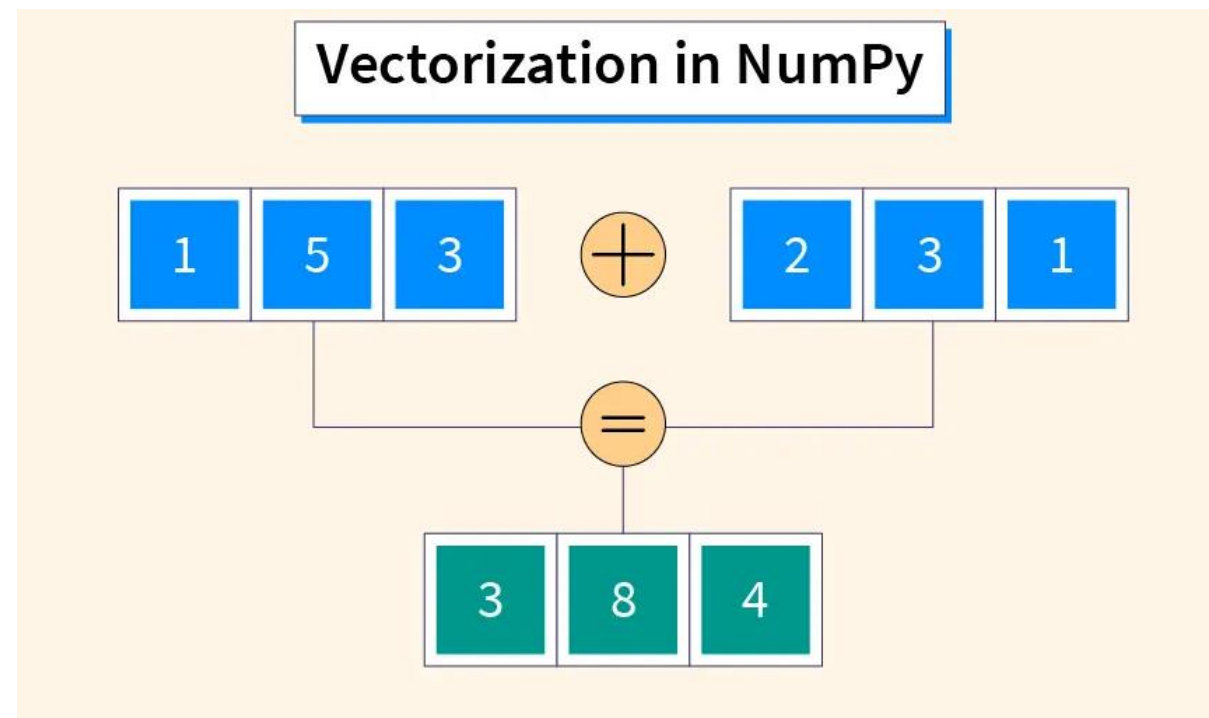
Vector: Objeto 1-dimensional

Matriz: Objeto 2-dimensional

Tensor: Objeto N-dimensional

Numpy ofrece además:

- **Vectorización:** operaciones matemáticas rápidas a nivel de arrays, sin bucles.
- Lectura/escritura.
- Operaciones de álgebra lineal, estadística, cálculo, y generación de valores aleatorios.



Fuente: scaler.com

2. LIBRERÍAS PYTHON: NUMPY, PANDAS, POLARS

Creación de arrays en numpy

- Función ***np.array()*** → crea un array manualmente, a partir de colecciones como listas y tuplas.

```
data = np.array([[1.5, -0.1, 3], [0, -3, 6.5]])
data1 = [6, 7.5, 8, 0, 1]
arr1 = np.array(data1)
```

- Atributos **shape**, **ndim**, y **dtype** permiten conocer el tamaño, número de dimensiones, y tipo de los elementos de un array.
- Más métodos para crear arrays particulares:

np.zeros(shape), np.ones(shape)	Crea un array de ceros (o unos), con las dimensiones especificadas en <i>shape</i>
np.full(shape, valor)	Crear un array con el valor especificado en todos sus elementos
np.identity(N)	Crear una matriz (2D) identidad: 1s en la diagonal principal, 0s en el resto.
np.arange(N)	Crea un array 1D con valores entre 0 y N-1
np.linspace(min, max, num)	Crea un array lineal 1D con valores equidistantes entre min y max

Operaciones aritméticas con arrays en NumPy:

- **Suma, diferencia, multiplicación, división y potenciación entre dos arrays de igual tamaño.**
- La operación se realiza aisladamente para cada elemento o posición de los arrays usados como operandos.

```
arr = np.array([[1., 2., 3.], [4., 5., 6.]])  
arr + arr
```

```
array([[ 2.,  4.,  6.], [ 8., 10., 12.]])
```

- Las operaciones entre un array y un escalar se extienden a todos los elementos del array:

```
arr ** 2
```

```
array([[ 1.,  4.,  9.], [16., 25., 36.]])
```

- La comparación entre arrays de igual tamaño devuelve un array de booleanos:

```
arr2 = np.array([[0., 4., 1.], [7., 2., 12.]])  
arr2 > arr
```

```
array([[False,  True, False], [ True, False,  True]])
```

Acceso y selección de elementos de un array:

- **Indexado sencillo 1D:** permite acceder a un solo elemento por índice y devuelve un valor escalar .

```
a = np.array([10, 20, 30])  
print(a[1])    # Mostrará 20
```

- **En arrays 2D o de dimensión superior, una indexación simple no devuelve un escalar sino un subarray de dimensión inferior.**

```
arr2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
arr2d[2]  
  
array([7, 8, 9])
```

- **Acceso por porciones (slicing):** permite seleccionar rangos de forma similar a las listas de Python.
- **CAUIDADO:** por razones de eficiencia, asignar el subarray a otra variable crea una vista lógica -no una copia física- del array original. Todo cambio posterior en el subarray afectará también al array original.

```
a = np.array([10, 20, 30, 40])  
b = a[1:3]  
b[0] = 99  
print(a)    # Mostrará [10 99 30 40]
```

Acceso y selección de elementos de un array:

- **Acceso por porciones (slicing):** aplicar slicing o combinaciones de slicing e índices individuales, permite realizar selecciones de arrays 2D y superiores.

```
arr2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
arr2d[:2]
```

1	2	3
4	5	6
7	8	9

```
arr2d[:2, 1:]
```

1	2	3
4	5	6
7	8	9

```
arr2d[:2, 2] # El array resultante será 1D
```

```
array([3, 6])
```

```
arr2d[:2, 1:] = 0 # Asignación sobre una selección
```

1	0	0
4	0	0
7	8	9

2. LIBRERÍAS PYTHON: NUMPY, PANDAS, POLARS

Indexado lógico o booleano:

- **Algo más complejo pero muy flexible y útil:** filtra elementos que cumplan una condición, la cual puede ser simple, compuesta, o incluso relacionada con los elementos de otro *array* relacionado.
- **Este tipo de indexado sí realiza una copia del array original como resultado de una asignación**

```
a = np.array([10, 20, 30, 40])
b = a[a > 25]
b[0] = 999
print(a)    # El array a se mantiene sin modificar: [10 20 30 40]
```

Ejemplo: datos de pacientes en dos arrays. Uno es una matriz con la edad, presión arterial y colesterol de cada paciente (fila), y el otro es un array 1D booleano que indica si el paciente es mayor de 50 años o no.

```
pacientes = np.array([
    [45, 120, 200],
    [60, 140, 240],
    [35, 110, 180],
    [70, 160, 260]
])
```

```
mayores_50 = np.array([False, True, False, True])
```

```
filtro = pacientes[mayores_50]
```

Dos modos de filtrar pacientes según la edad supere los 50 años. El inferior es más complejo pero no necesita el array 1D adicional

```
filtro = pacientes[pacientes[:, 0] > 50]
```

```
array([[ 60, 140, 240],
       [ 70, 160, 260]])
```


Indexado lógico o booleano:

- **Algo más complejo pero muy flexible y útil:** filtra elementos que cumplan una condición, la cual puede ser simple, compuesta, o incluso relacionada con los elementos de otro *array* relacionado.
- **Este tipo de indexado sí realiza una copia del array original como resultado de una asignación**

```
a = np.array([10, 20, 30, 40])
b = a[a > 25]
b[0] = 999
print(a)      # El array a se mantiene sin modificar: [10 20 30 40]
```

Ejemplo: datos de pacientes en dos arrays. Uno es una matriz con la edad, presión arterial y colesterol de cada paciente (fila), y el otro es un array 1D booleano que indica si el paciente es mayor de 50 años o no.

```
pacientes = np.array([
    [45, 120, 200],
    [60, 140, 240],
    [35, 110, 180],
    [70, 160, 260]
])
```

```
mayores_50 = np.array([False, True, False, True])
```

MÁS "DIFÍCIL" TODAVÍA: filtrar pacientes con edad > 50 y presión arterial > 150.

```
array([[ 70, 160, 260]])
```

```
filtro = pacientes[(pacientes[:, 0] > 50) & (pacientes[:, 1] > 150)]
```

Transposición de arrays e intercambio de ejes

- La función *reshape()* permite cambiar la forma de un array, siempre que la totalidad de elementos se mantenga.

```
arr = np.arange(15).reshape((3, 5))
```

```
array([[ 0,  1,  2,  3,  4],  
       [ 5,  6,  7,  8,  9],  
       [10, 11, 12, 13, 14]])
```

- El atributo *T* permite obtener la transpuesta de de una matriz

```
arr.T
```

```
array([[ 0,  5, 10],  
       [ 1,  6, 11],  
       [ 2,  7, 12],  
       [ 3,  8, 13],  
       [ 4,  9, 14]])
```

- El producto interior entre matrices (no a nivel de elementos), se efectúa de dos posibles formas:

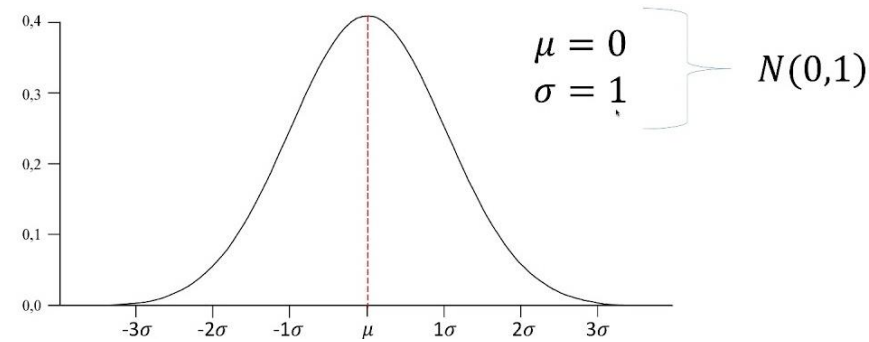
```
np.dot(arr.T, arr) # Producto interior entre matrices 3x5 y 5x3  
arr.T @ arr # Otra posible forma, con el operador @
```

```
array([[39, 20, 12],  
       [20, 14,  2],  
       [12,  2, 10]])
```

Generación de números pseudoaleatorios

- Se hace con el módulo *random* de *numpy*. Este ejemplo genera una matrix 4x4 con números que siguen una distribución normal estándar:

```
samples = np.random.standard_normal(size=(4, 4))
```



```
array([[ -0.2047,  0.4789, -0.5194, -0.5557],  
       [ 1.9658,  1.3934,  0.0929,  0.2817],  
       [ 0.769 ,  1.2464,  1.0072, -1.2962],  
       [ 0.275 ,  0.2289,  1.3529,  0.8864]])
```

- En código reproducible, se suele adoptar la práctica de personalizar la semilla de generación de números aleatorios.

```
rng = np.random.default_rng(seed=12345)  
data = rng.integers(0, 10, size=(4, 2))  
data
```

```
array([[8, 8],  
       [6, 6],  
       [3, 9],  
       [6, 7]])
```

Métodos estadísticos

- En arrays multidimensionales, usando el argumento axis especificamos si queremos un estadístico calculado separadamente en cada dimensión (e.g. fila, columna).

```
arr = np.array([[1, 2, 3],
               [4, 5, 6]])
```



```
[1  2  3]
[4  5  6]
```

- Con axis=1, se opera atravesando las columnas → **estadístico para cada fila**

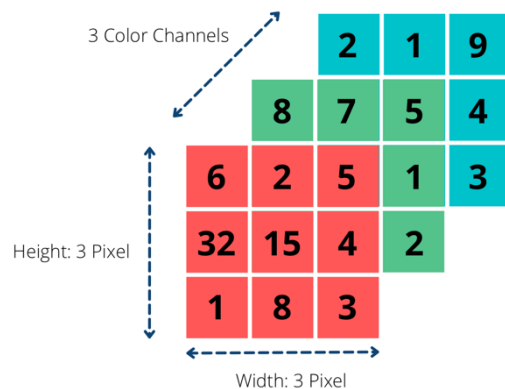
```
np.mean(arr, axis=1)
```

```
array([2., 5.])
```

- Con axis=0, se opera atravesando las filas → **estadístico para cada columna**

```
np.median(arr, axis=0)
```

```
array([2.5, 3.5, 4.5])
```



En Tensores 3D, como imágenes con valores RGB para cada píxel, también se puede agregar. Por ejemplo, la agregación de la tercera dimensión (valores RGB) en cada píxel de una imagen 3 x 3 x 2 sería:

```
fusion_colores = np.mean(arr, axis=2)
```

Y si deseamos agregar a través de varias dimensiones:

```
fusion_colores_columnas = np.mean(arr, axis=(1,2))
```

PANDAS: Ejemplo introductorio de Pandas

- `pd.read_csv()` crea un `DataFrame` -la principal estructura de Pandas para datos estructurados o tabulares- a partir de un fichero.
- Atributos básicos del `DataFrame` como *shape*, *columns*, permiten conocer las dimensiones y columnas del dataset.

Muestra los primeros cinco
elementos del DataFrame

```
import pandas as pd

# Cargar el dataset
df = pd.read_csv("50_Startups.csv")

# Exploración básica
print(df.shape)
print(df.columns)
df.head()
print(df.describe())

# Acceso a una Series (estructura 1D)
gastos_marketing = df["Marketing Spend"]
print(gastos_marketing.head())

# Guardar el DataFrame
df.to_csv("50_Startups_copia.csv", index=False)
```

Muestra un resumen
estadístico sencillo del dataset

Selección de datos en un DataFrame de Pandas

- Selección de una columna (atributo del dataset):

```
df = pd.read_csv("50_Startups.csv")
```

```
marketing = df["Marketing Spend"]
```

Se guarda el atributo como un objeto Series

```
marketing_df = df[["Marketing Spend"]]
```

Se guarda el atributo como un DataFrame (bidimensional) de una sola columna

- Selección de una fila (instancia u observación del dataset):

```
print(df.iloc[0])
```

```
print(df.iloc[[0]])
```

Devuelve un Series cuyos índices son los nombres de las columnas del DataFrame *df*

Devuelve un DataFrame de una sola fila

```
type(marketing)  
type(marketing_df)
```

```
<class 'pandas.core.series.Series'>  
<class 'pandas.core.frame.DataFrame'>
```

Creando un DataFrame a partir de ...

Colecciones nativas de Python:

- Lista → **Series**
- Lista de listas → **DataFrame**
- Diccionario → **Series**
- Diccionario de listas → **DataFrame**
- Lista de diccionarios → **DataFrame**

```
[10, 20, 30]

[[1, 'Ana'], [2, 'Luis']]

{"a": 1, "b": 2, "c": 3}

{
  "Nombre": ["Ana", "Luis"],
  "Edad": [28, 35],
  "Ciudad": ["Madrid", "Sevilla"]
}

[{"Nombre": "Ana", "Edad": 28},
 {"Nombre": "Luis", "Edad": 35}]
```

Arrays de NumPy

- Array1D → **Series** (también se puede convertir en un DataFrame de una columna -por defecto- o una fila)
- Array2D → **DataFrame**

¿Y cómo funcionaría al revés?

Crear *arrays* y colecciones nativas a partir de datasets de Pandas también es posible, según las condiciones y naturaleza de los datos.

- El caso más simple es pasar un *Series* a una lista sencilla:

```
serie = pd.Series([10, 20, 30])  
lista = serie.tolist()
```

- Pasar un *Series* a un diccionario, hace que las claves del diccionario sean los índices del *Series*:

- Pasar un *Series* a un array 1D:

```
arr = serie.to_numpy()  
# También funciona en un df
```

```
serie.to_dict()
```

Algo más complejo pero muy útil es pasar un **DataFrame** (o parte del mismo) a una **lista de diccionarios**, o bien un **diccionario de listas**:

```
df = pd.read_csv("50_Startups.csv")  
Dict_listas = df[:3].to_dict(orient="list")
```

```
df = pd.read_csv("50_Startups.csv")  
lista_dicts = df[:3].to_dict(orient="records")
```


Selección de datos en Pandas (I)

Tipo de acceso	Sintaxis	Ejemplo	Resultado
Columna de un <i>df</i> (una)	<code>df["col"]</code>	<code>df["A"]</code>	<i>Series</i> con la columna "A"
Columnas de un <i>df</i> (varias)	<code>df[["A", "B"]]</code>	<code>df[["A", "B"]]</code>	<i>DataFrame</i> con columnas A y B
Fila por posición	<code>df.iloc[i]</code>	<code>df.iloc[0]</code>	<i>Series</i> con la fila 0
Fila por etiqueta	<code>df.loc["f1"]</code>	<code>df.loc["f1"]</code>	<i>Series</i> con la fila "f1"
Filas (varias)	<code>df.iloc[[i, j]]</code>	<code>df.iloc[[0, 2]]</code>	<i>DataFrame</i> con filas 0 y 2
Celda [fila, col]	<code>df.loc[i, "col"]</code> <code>df.iloc[i, j]</code>	<code>df.loc["f1", "A"]</code> <code>df.iloc[0, 1]</code>	Valor escalar

Selección de datos en Pandas (II)

Tipo de acceso	Sintaxis	Ejemplo o caso equivalente	Resultado
Slicing de filas	<code>df[a:b]</code>	<code>df[1:3]</code>	Filas 1 y 2
Slicing por posición	<code>df.iloc[a:b, :]</code>	<code>df.iloc[1:3, :]</code>	Filas 1 y 2
Slicing por etiquetas	<code>df.loc["f1":"f3"]</code>	<code>df.loc["f1":"f2"]</code>	Rango inclusivo por índices
Filtro booleano (df)	<code>df[condición]</code>	<code>df[df["A"] > 15]</code>	Filas donde $A > 15$
Filtro booleano (Series)	<code>serie[condición]</code>	<code>s[s > 0]</code>	Valores mayores a 0

Aunque no es habitual, puede hacerse *slicing* para seleccionar un rango de columnas consecutivas, debiendo hacerse así:

```
df.iloc[:, 1:3] # Todas las filas, columnas 1 y 2 (no incluye la 3)
```

Limpieza, transformación de datos, e ingeniería de características en Pandas

Las labores de limpieza y transformación de datos más habituales en dataset que están siendo preprocesados con Pandas incluyen:

- **Manejo de valores perdidos:** se pueden eliminar instancias completas que presenten valores perdidos, o bien se pueden rellenar (imputar) estos valores.

```
housing_df.isna().sum()
```

```
housing_df_dropna = housing_df.dropna(subset=["total_bedrooms"])
```

Se eliminan las instancias donde no hay valor para "total_bedrooms"

```
mediana = housing_df["total_bedrooms"].median()  
housing_df["total_bedrooms"] = housing_df["total_bedrooms"].fillna(mediana)
```

Se rellenan los valores perdidos del atributo "total_bedrooms" con la mediana estadística de los valores existentes de ese atributo.



longitude	0
latitude	0
housing_median_age	0
total_rooms	0
total_bedrooms	207
population	0
households	0
median_income	0
median_house_value	0
ocean_proximity	0

```
df.dropna(axis=1, inplace=True)
```

Elimina todos los atributos (columnas) que contengan al menos un NaN (valor perdido)

Limpieza, transformación de datos, e ingeniería de características en Pandas

Las labores de limpieza y transformación de datos más habituales en dataset que están siendo preprocesados con Pandas incluyen:

- **Manejo de duplicados:** instancias que están duplicadas al completo (idénticos valores en todos sus atributos) pueden eliminarse mediante.

Otro argumento útil es keep:

- **keep='first'** mantiene el primero de los registros repetidos.
- **keep = 'last'** mantiene el último.
- **keep = False** los elimina todos.

```
housing_df.drop_duplicates(inplace=True)
```

`inplace = True` **aplica los cambios directamente en el propio DataFrame**

- **Codificación de atributos categóricos con one-hot encoding:** gracias a `pd.get_dummies()`, esto se hace de forma extremadamente sencilla.

Color	One-hot encoding		
	d1	d2	d3
Red	1	0	0
Green	0	1	0
Blue	0	0	1

Escalado de atributos numéricos: Normalización y Estandarización

El escalado es útil y necesario en situaciones donde tenemos varios atributos numéricos cuyos valores se mueven en escalas muy diferentes entre sí.

- Además, varios modelos de *machine learning* requieren un escalado de los datos para funcionar de forma eficaz.

Pandas permite realizar dos escalados de forma manual: **normalización** y **estandarización**.

- **Normalización (Min-Max):** se normalizan los valores de un atributo al intervalo [0,1], siguiendo esta fórmula. Se conserva la forma original de la distribución de los datos. Recomendable para datos uniformemente distribuidos.

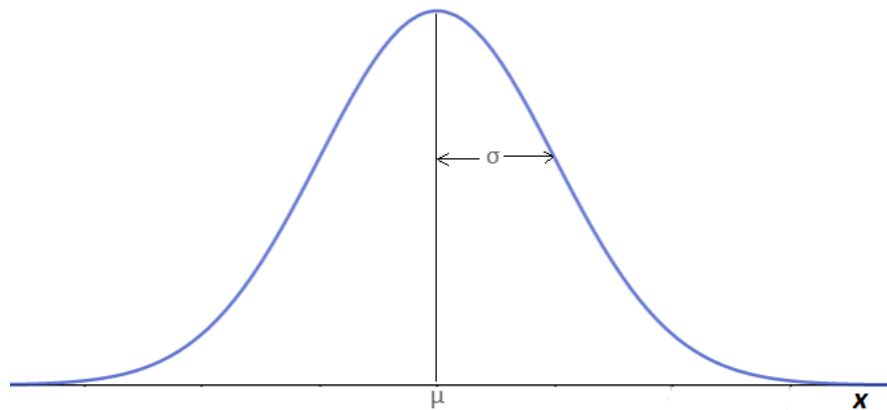
$$\bar{x} = \frac{x - \min}{\max - \min}$$

```
df_minmax = (df_num - df_num.min()) / (df_num.max() - df_num.min())
```

Entiéndase **df_num** como el DataFrame resultante de seleccionar aquellos atributos numéricos que se quiera normalizar

Estandarización (Z-score): este proceso no solo escala los datos, sino que los centra en torno a la media.

- Usar si los datos están normalmente distribuidos (mayoría en torno a un rango central de valores, y minorías en rangos bajos y altos) o hay *outliers* leves o moderados.
- La estandarización es más útil si las escalas varían mucho entre atributos.



$$z = \frac{x - \mu}{\sigma}$$

μ = Media

σ = Desviación típica

```
df_standardized = (df_num - df_num.mean()) / df_num.std()
```

2. LIBRERÍAS PYTHON: NUMPY, PANDAS, POLARS

La librería **scikit-learn** proporciona métodos más automatizados y fiables en la práctica para realizar diferentes tipos de escalado, no solo min-max y estandarización:

- Estandarización:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df_standardized = pd.DataFrame(scaler.fit_transform(df_num), columns=atributos_numericos)
```

- Normalización min-max:

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df_minmax = pd.DataFrame(scaler.fit_transform(df_num), columns=atributos_numericos)
```

- Escalado robusto, mediante mediana y rango intercuartílico –menos sensible a *outliers* muy alejados.

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
df_robust = pd.DataFrame(scaler.fit_transform(df_num), columns=atributos_numericos)
```

Numpy también permite escalado mediante la **transformación logarítmica**:

- Útil para datos muy sesgados hacia la derecha (*right-skewed distribution*).

```
import numpy as np  
  
df_log = np.log1p(df_num)
```



RECOMENDACIÓN: Guarda el resultado de escalar atributos numéricos en un nuevo dataframe, ya que seguramente después quieras volver a fusionarlos con el resto de atributos que no han formado parte del escalado.

```
# Paso 1: Del DataFrame original, eliminar atributos numéricos  
# que han sido escalados  
df_no_numeric = housing_df.drop(columns=[...])  
  
# Paso 2: Concatenar el DataFrame resultante del paso 1 con el de  
# atributos ya escalados  
df_final = pd.concat([df_no_numeric, housing_df_num], axis=1)
```

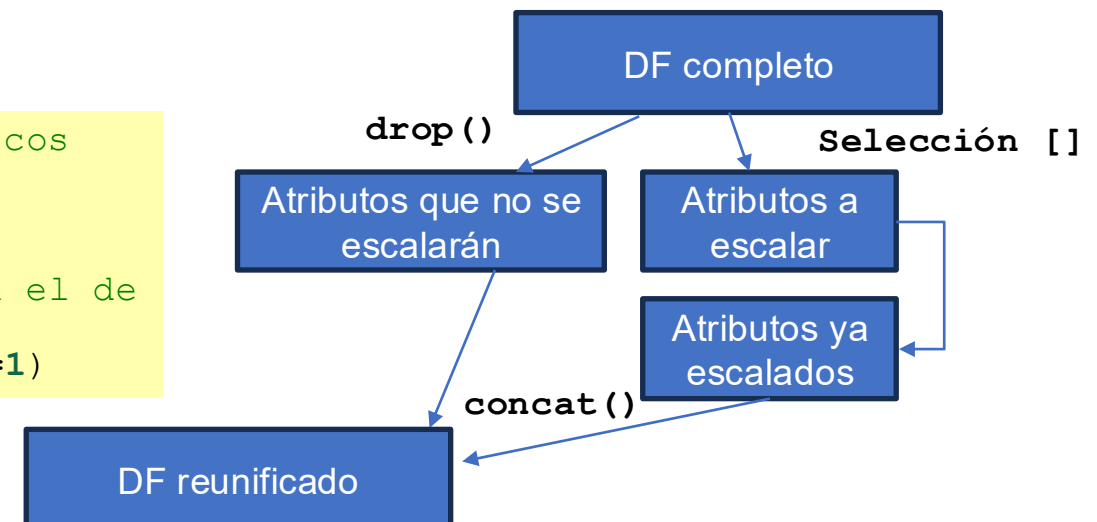


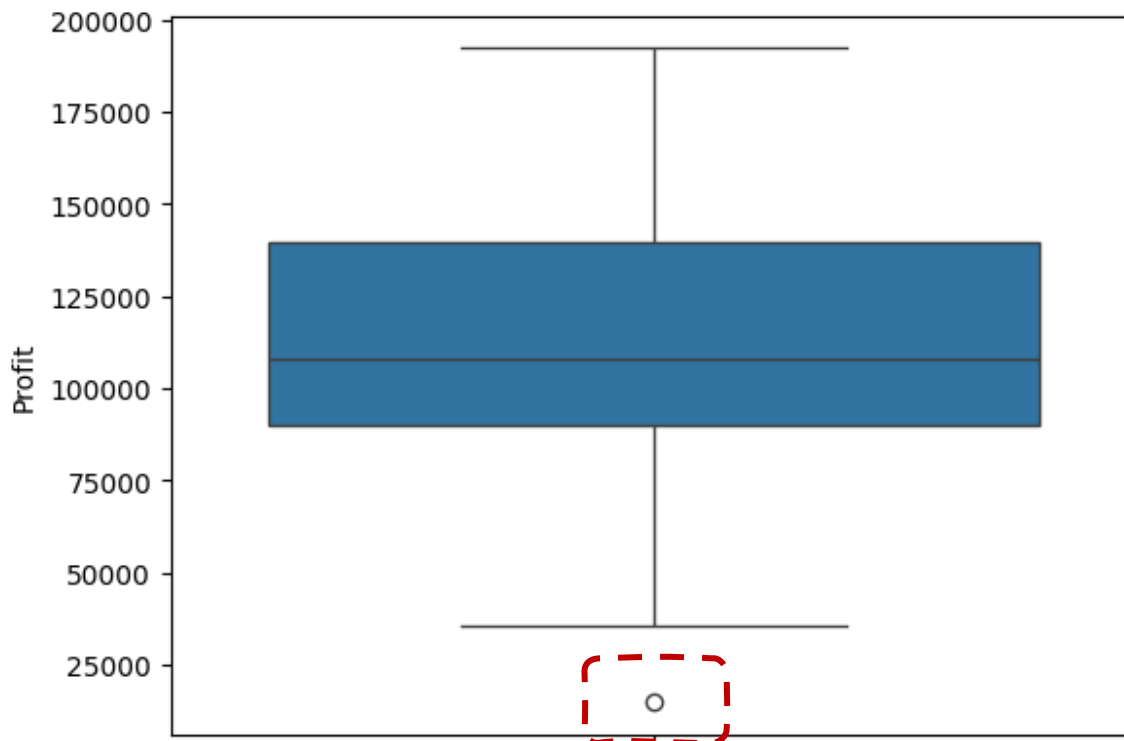
Tabla-resumen: guía práctica para decidir si escalar o no atributos, cuando, y cómo

Tipo de Atributo	Transformación Recomendable	¿Por qué?
Precios altos, ingresos, superficie (grandes magnitudes)	<code>StandardScaler</code> o <code>MinMaxScaler</code>	Reduce varianza de los datos y mejora convergencia de modelos predictivos
Atributos muy sesgados (ej. Ingresos de una gran población)	<code>np.log1p()</code>	Para acercar a una distribución normal
Porcentajes o proporciones (entre 0 y 100)	A veces <code>MinMaxScaler</code> , o ninguna	Ya están en un rango fijo. La decisión de normalizar a 0-1 dependerá de la naturaleza de otros atributos relevantes.
Binarios (0/1)	Ninguna	Ya están normalizados
Recuentos enteros (habitaciones, personas, pisos)	<code>StandardScaler</code> (o <code>RobustScaler</code> si hay outliers)	Homogeneiza recuentos dispares entre sí
Identificadores numéricos (ej. ID de casa)	⚠ No escalar ni incluir en el modelo o análisis	No contienen información útil para el análisis

Manejo de valores atípicos (*outliers*)

Outliers son datos atípicos en un conjunto de datos, con valores anormalmente alejados del resto para uno o varios atributos.

- El método más intuitivo para detectarlos es por realizar una visualización como un *boxplot* del atributo a investigar, o un diagrama de dispersión (*scatter plot*) entre dos atributos.



¿Cómo eliminarlos? Método del rango intercuartílico

```
Q1 = df['Profit'].quantile(0.25)
Q3 = df['Profit'].quantile(0.75)

# Calcular el IQR
IQR = Q3 - Q1

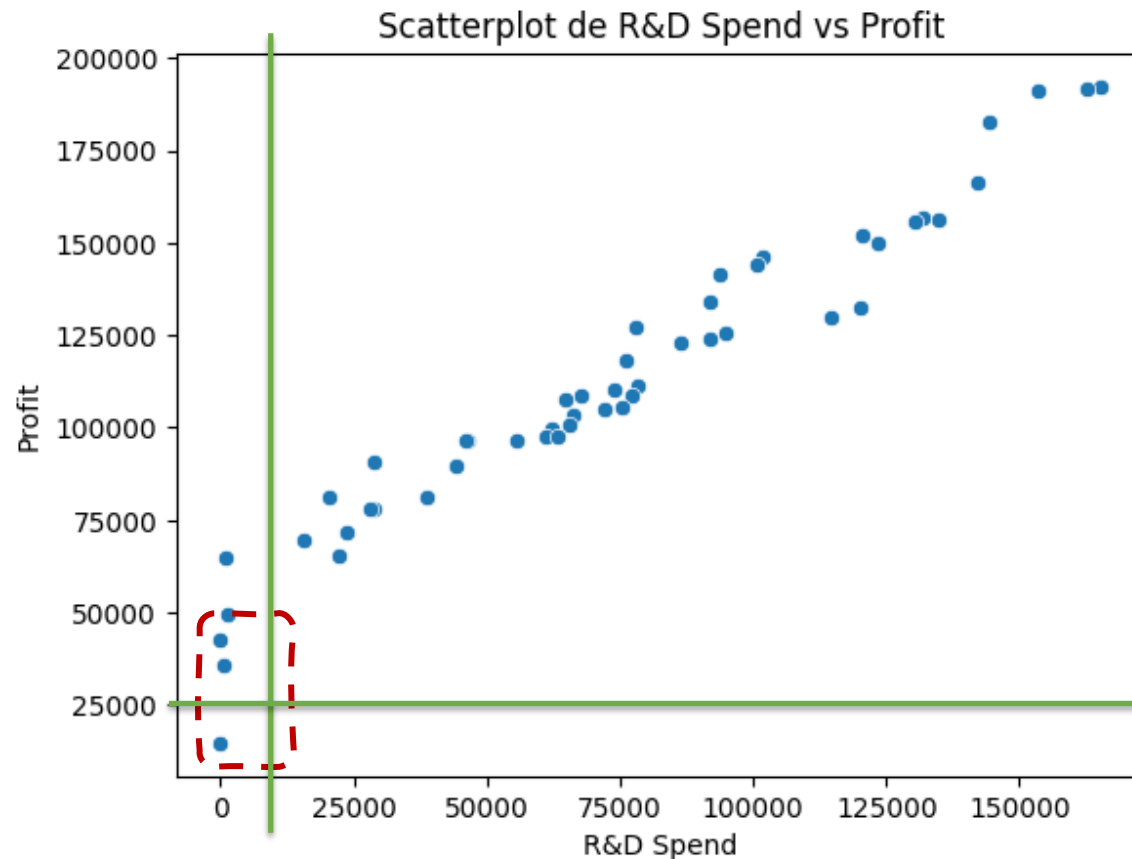
# Definir los límites inferior y superior
limite_inf = Q1 - 1.5 * IQR
limite_sup = Q3 + 1.5 * IQR

# Filtrar los datos sin outliers
df_filtered = df[(df['Profit'] >= limite_inf) &
                 (df['Profit'] <= limite_sup)]
df_filtered.shape
```

2. LIBRERÍAS PYTHON: NUMPY, PANDAS, POLARS

Manejo de valores atípicos (*outliers*)

También pueden detectarse a partir de un *scatterplot* en cuyo caso se podría visualmente decidir qué regiones filtrar del *dataset* y cuáles excluir, todo ello de forma manual:



¿Cómo eliminarlos? Selección manual de los datos que queremos mantener

```
df_filtered = df[(df['R&D Spend'] > 5000) &
                  (df['Profit'] > 25000)]
```

Este método es más manual, subjetivo, y bastante sujeto al “prueba y error”.

PANDAS vs POLARS

Característica	Pandas 🐼	Polars 🐻❄️
Lenguaje base	Python / C	Rust (Seguridad de memoria)
Modelo de ejecución	Mono-hilo (Single-core)	Multi-hilo nativo (Paralelismo)
Gestión de Memoria	Copia datos frecuentemente	Zero-copy (Apache Arrow)
Evaluación de Operaciones	Eager (Inmediata)	Lazy (Diferida/Planificada)

Pandas es el estándar y resulta fabuloso para datasets no demasiado grandes. Sin embargo, procesa las operaciones en un único núcleo del procesador.

Polars, escrito en **Rust**, divide el trabajo entre todos los núcleos de vuestro ordenador automáticamente. Además, usa **Apache Arrow**, tecnología que facilita el intercambio de datos entre herramientas de modo instantáneo y sin consumir RAM extra.

EVALUACIÓN PEREZOSA (Lazy) EN POLARS

- Con Pandas, se ejecuta cada línea de procesamiento de datos al instante.
- Con Polars, se define un "plan de ejecución", pero este no se lleva a cabo hasta que pidamos el resultado final a posteriori, con el método *collect()*

```
import polars as pl

# 1. Creamos el plan (No se ejecuta nada todavía)
plan = (
    pl.scan_csv("gran_auditoria.csv") # Lectura dataset: Escaneo inteligente
    .filter(pl.col("importe") > 5000) # Filtro
    .select(["id", "fecha", "importe"]) # Selección de columnas
)

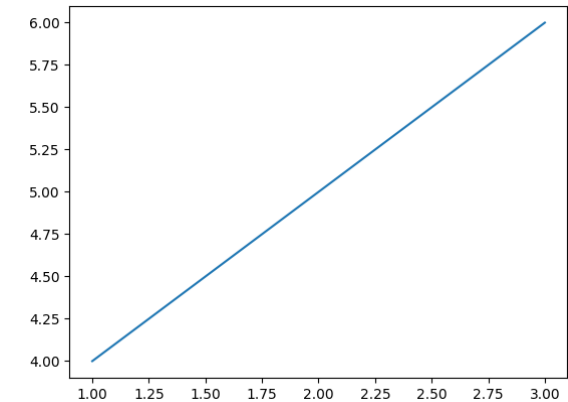
# 2. El optimizador reordena los pasos para que sea más rápido
# 3. Ejecutamos al final
df_final = plan.collect()
```

3. VISUALIZACIÓN DE DATOS: MATPLOTLIB, SEABORN, PLOTLY

Matplotlib

Librería a bajo nivel, muy flexible pero verbosa (difícil de “memorizar” su uso).
Útil para un control total sobre el gráfico a generar y su configuración

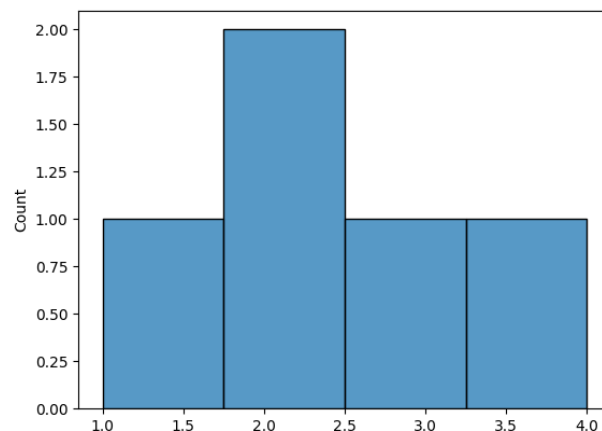
```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [4, 5, 6])
plt.show()
```



Seaborn

Más estética y con enfoque en gráficos estadísticos elegantes a partir de código menos denso.

```
import seaborn as sns
sns.histplot([1, 2, 2, 3, 4])
```

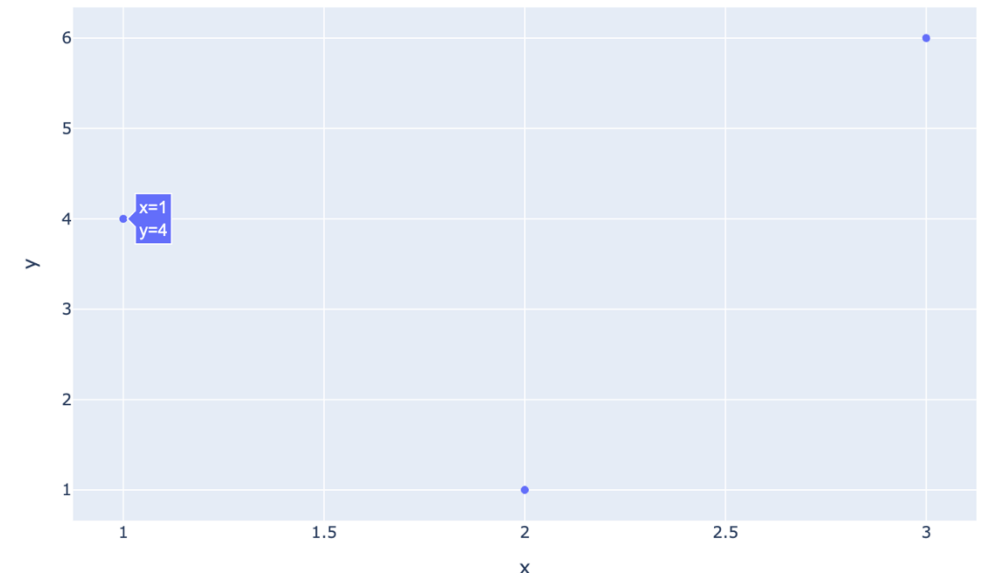


Plotly

Permite visualizaciones interactivas:

zoom, hover, clic

Ideal para construir **dashboards**



Seaborn es una librería diseñada para usarse en conjunción con **matplotlib** para simplificar la creación y personalización de distintos tipos de gráficos de visualización.

- Veremos una comparativa de creaciones de gráficos similares con ambas librerías.

Gráfico de barras - matplotlib

```
counts = housing_df['ocean_proximity'].value_counts()
plt.bar(counts.index, counts.values, color='teal')
plt.title('Distribución de viviendas según cercanía al océano')
plt.xlabel('Categoría de proximidad')
plt.ylabel('Cantidad de registros')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

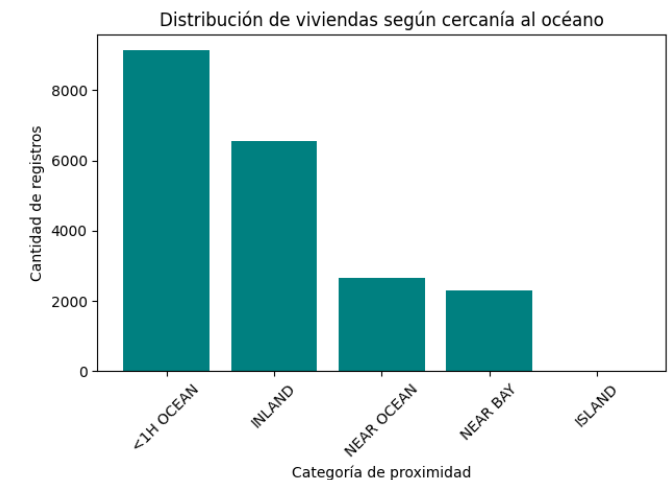
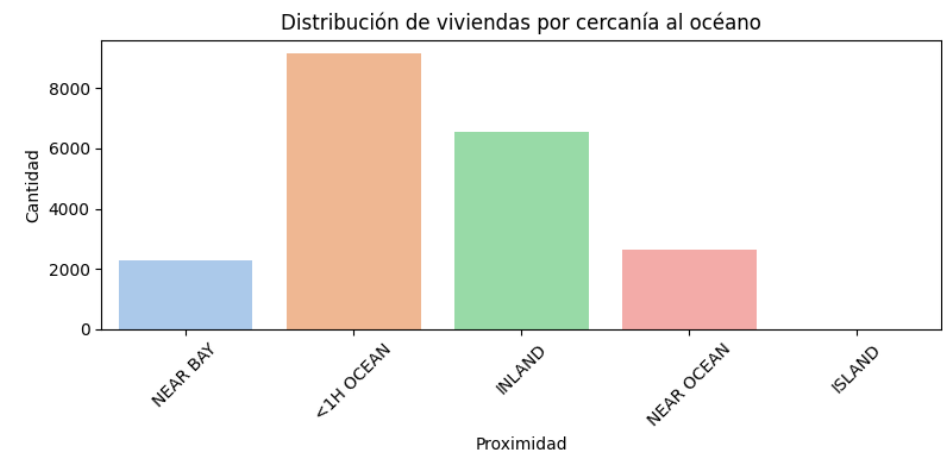


Gráfico de barras - seaborn

```
plt.figure(figsize=(8, 4))
sns.countplot(data=housing_df, x='ocean_proximity', palette='pastel')
plt.title('Distribución de viviendas por cercanía al océano')
plt.xlabel('Proximidad')
plt.ylabel('Cantidad')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



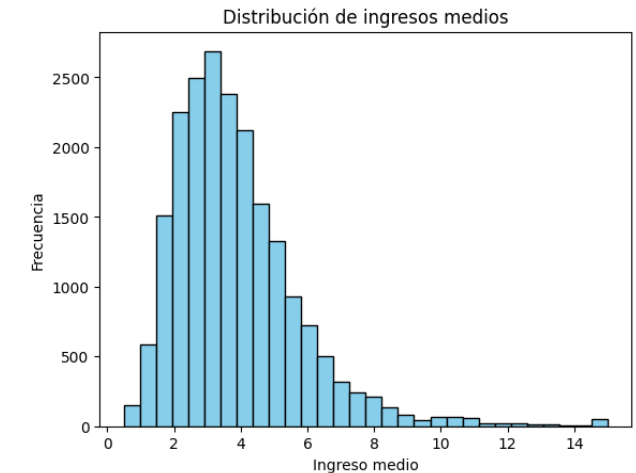
3. VISUALIZACIÓN DE DATOS: MATPLOTLIB, SEABORN, PLOTLY

Seaborn es una librería diseñada para usarse en conjunción con **matplotlib** para simplificar la creación y personalización de distintos tipos de gráficos de visualización.

- Veremos una comparativa de creaciones de gráficos similares con ambas librerías.

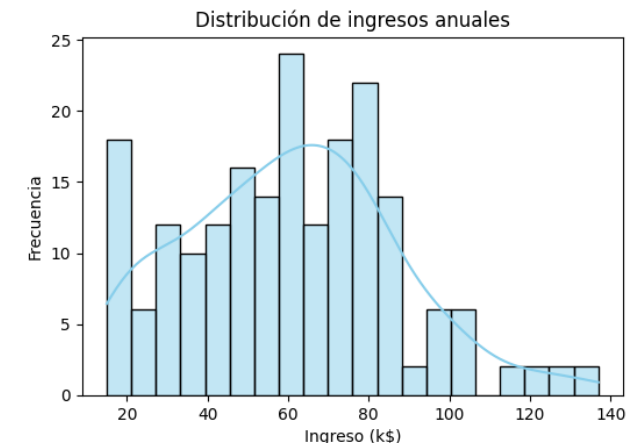
Histograma de columnas - matplotlib

```
plt.hist(housing_df['median_income'], bins=30, color='skyblue',
         edgecolor='black')
plt.title('Distribución de ingresos medios')
plt.xlabel('Ingreso medio')
plt.ylabel('Frecuencia')
plt.show()
```



Histograma de columnas - seaborn

```
plt.figure(figsize=(6, 4))
sns.histplot(mall_df['Annual Income (k$)'], bins=20,
             kde=True, color='skyblue')
plt.title('Distribución de ingresos anuales')
plt.xlabel('Ingreso (k$)')
plt.ylabel('Frecuencia')
plt.show()
```



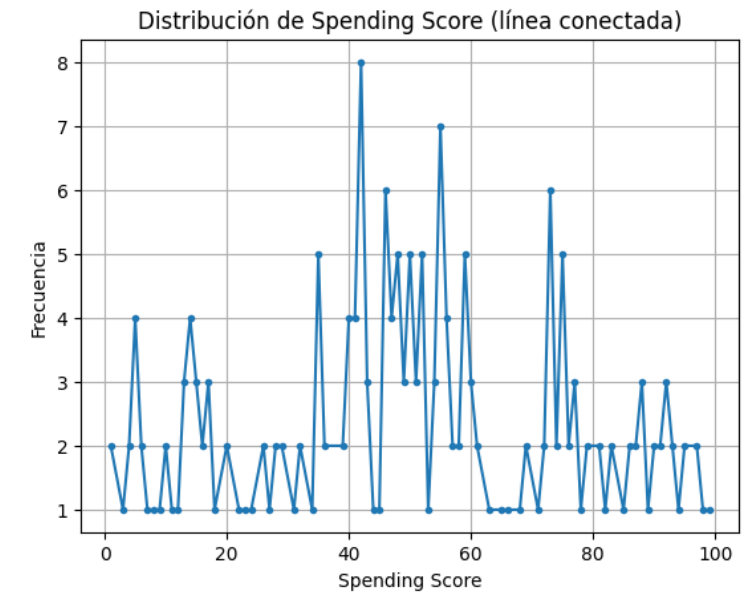
3. VISUALIZACIÓN DE DATOS: MATPLOTLIB, SEABORN, PLOTLY

Histograma de líneas - matplotlib

```
data = mall_df['Spending Score (1-100)'].dropna()

# Contar frecuencias directamente
counts = data.value_counts().sort_index() # En esta serie, los posibles
valores del atributo pasan a ser los índices

# Dibujar una línea que conecta los puntos
plt.plot(counts.index, counts.values, marker='.', linestyle='-')
plt.title('Distribución de Spending Score (línea conectada)')
plt.xlabel('Spending Score')
plt.ylabel('Frecuencia')
plt.grid(True)
plt.show()
```



Histograma de líneas - seaborn

```
data = mall_df['Spending Score (1-100)'].dropna()
counts = data.value_counts().sort_index()

plt.figure(figsize=(6, 4))
sns.lineplot(x=counts.index, y=counts.values, marker='.')
plt.title('Distribución de Spending Score (línea conectada)')
plt.xlabel('Spending Score')
plt.ylabel('Frecuencia')
plt.grid(True)
plt.show()
```

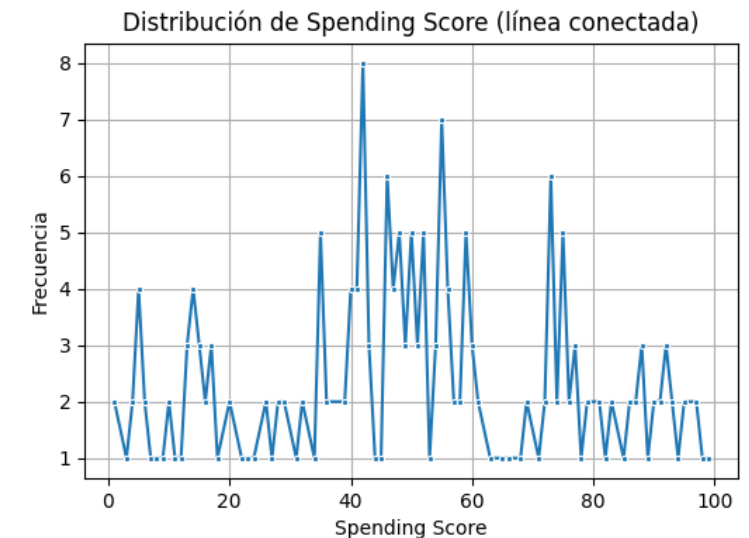


Diagrama de sectores – matplotlib (seaborn no ofrece un enfoque propio para gráficos de sectores)

```
gender_counts = mall_df['Gender'].value_counts()
plt.pie(gender_counts.values, labels=gender_counts.index, autopct='%1.1f%%', startangle=90)
plt.title('Clientes por género')
plt.axis('equal')
plt.show()
```

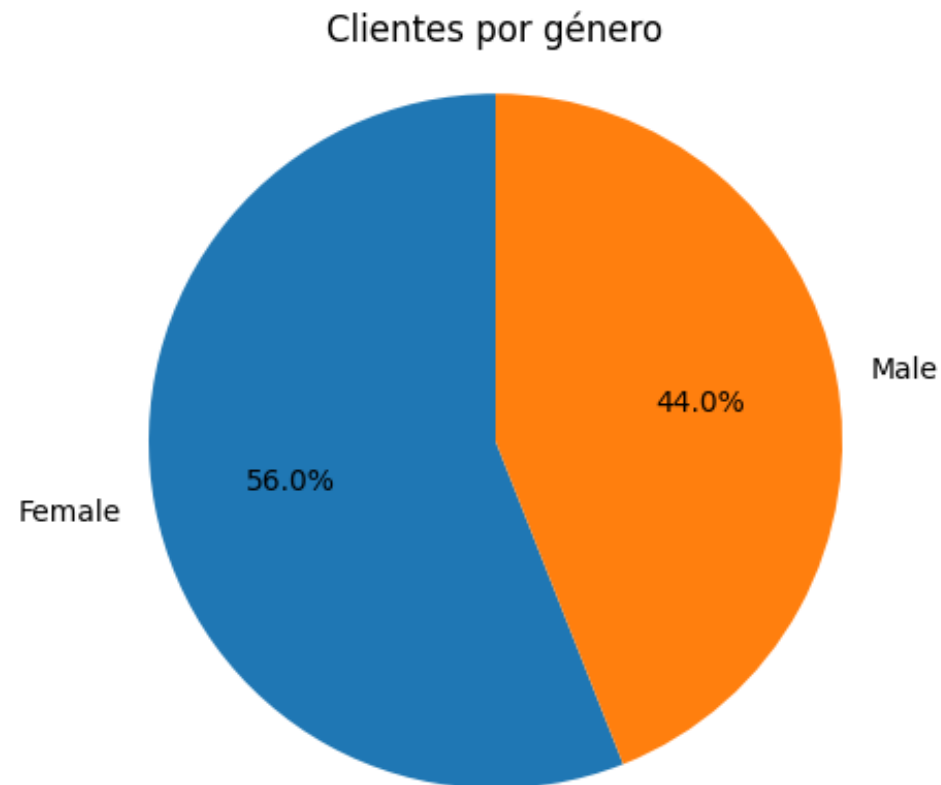


Gráfico de líneas - matplotlib

```
cta_january_2024 = cta_january_2024.sort_values('service_date')

plt.figure(figsize=(10, 5))
plt.plot(cta_january_2024['service_date'], cta_january_2024['total_rides'],
marker='o', linestyle='-')
plt.title('Embarques diarios en CTA - Enero 2024')
plt.xlabel('Fecha')
plt.ylabel('Total de embarques')
plt.xticks(rotation=45)
plt.tight_layout()
plt.grid(True)
plt.show()
```

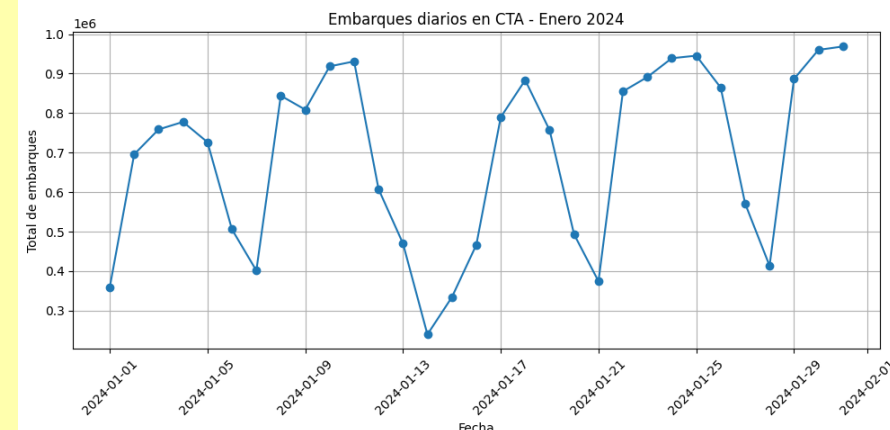
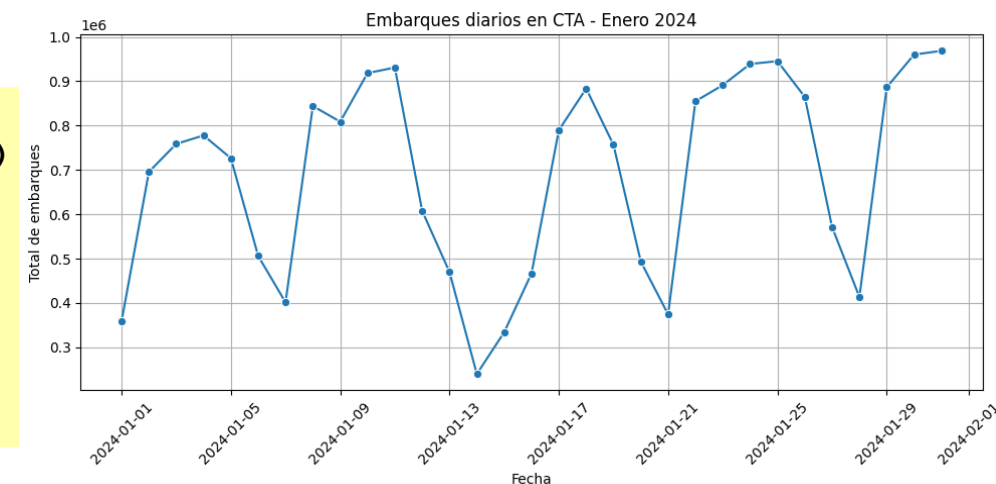


Gráfico de líneas - seaborn

```
plt.figure(figsize=(10, 5))
sns.lineplot(data=cta_sample, x='service_date', y='total_rides', marker='o')
plt.title('Embarques diarios en CTA - Enero 2024')
plt.xlabel('Fecha')
plt.ylabel('Total de embarques')
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```



3. VISUALIZACIÓN DE DATOS: MATPLOTLIB, SEABORN, PLOTLY

Diagrama de dispersión (scatterplot) - matplotlib

```
plt.scatter(housing_df['median_income'], housing_df['median_house_value'],
            alpha=0.5)
plt.title('Ingreso medio vs Valor vivienda')
plt.xlabel('Ingreso medio')
plt.ylabel('Valor de vivienda')
plt.show()
```

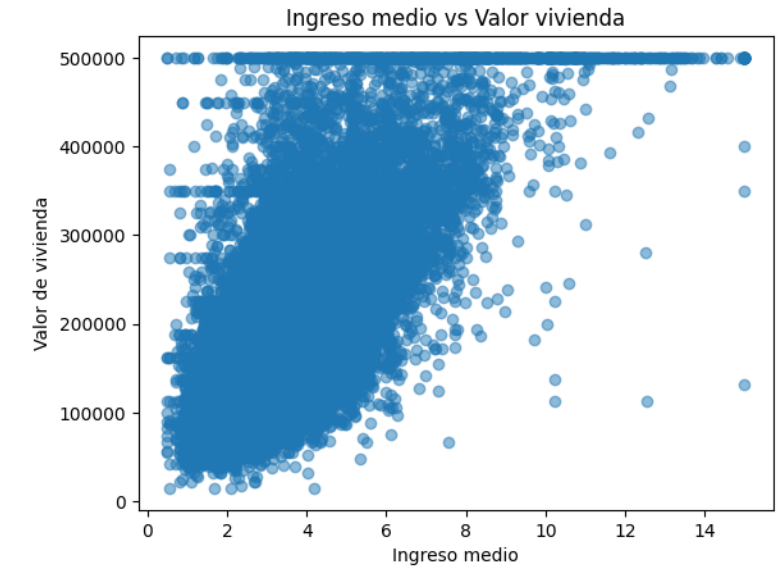


Gráfico de líneas - seaborn

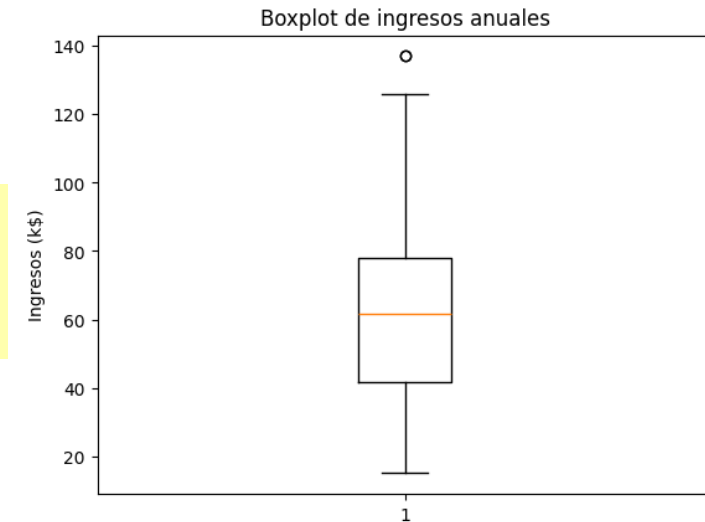
```
plt.figure(figsize=(6, 5))
sns.scatterplot(data=mall_df, x='Annual Income (k$)', y='Spending Score (1-100)',
               hue='Gender', palette='Set2')
plt.title('Spending Score vs Annual Income')
plt.xlabel('Ingreso anual (k$)')
plt.ylabel('Spending Score')
plt.legend()
plt.show()
```



3. VISUALIZACIÓN DE DATOS: MATPLOTLIB, SEABORN, PLOTLY

Boxplot - matplotlib

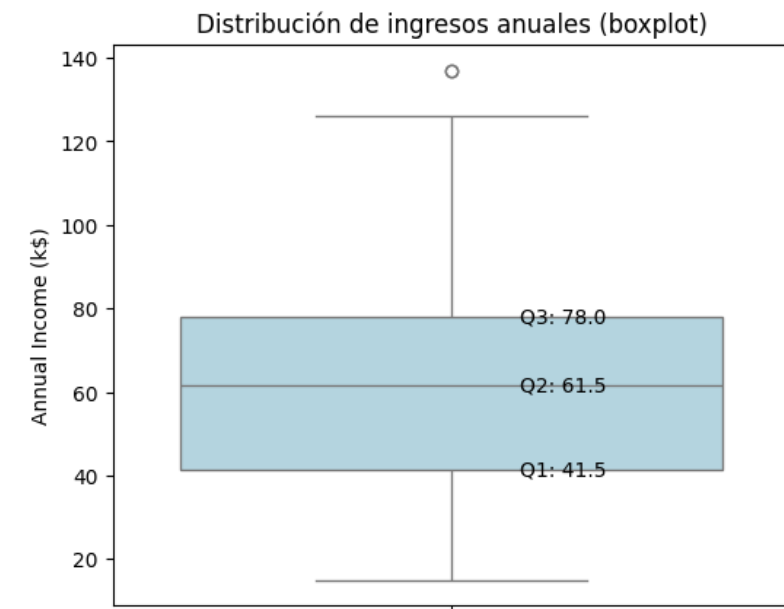
```
plt.boxplot(mall_df['Annual Income (k$)'])
plt.title('Boxplot de ingresos anuales')
plt.ylabel('Ingresos (k$)')
plt.show()
```



Boxplot - seaborn

```
plt.figure(figsize=(6, 5))
sns.boxplot(data=mall_df, y='Annual Income (k$)', color='lightblue')
plt.title('Distribución de ingresos anuales (boxplot)')

plt.show()
```

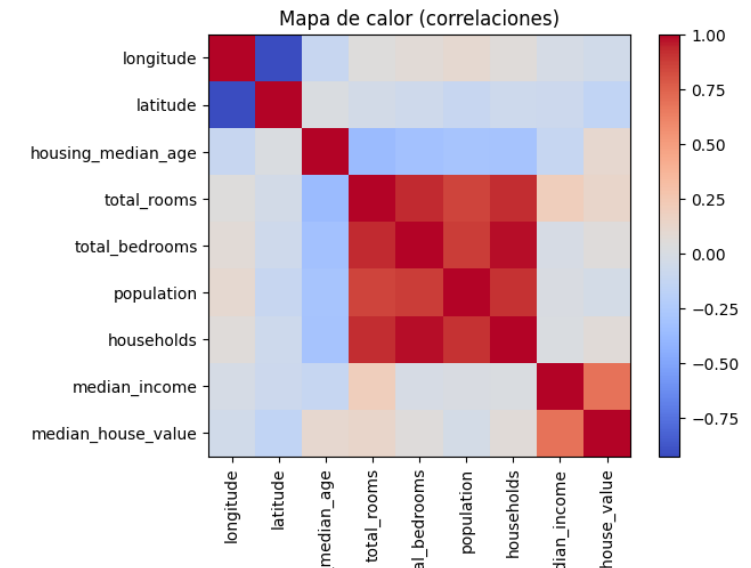


3. VISUALIZACIÓN DE DATOS: MATPLOTLIB, SEABORN, PLOTLY

Mapa de calor - matplotlib

```
numeric_housing = housing_df.select_dtypes(include='number')
corr = numeric_housing.corr()

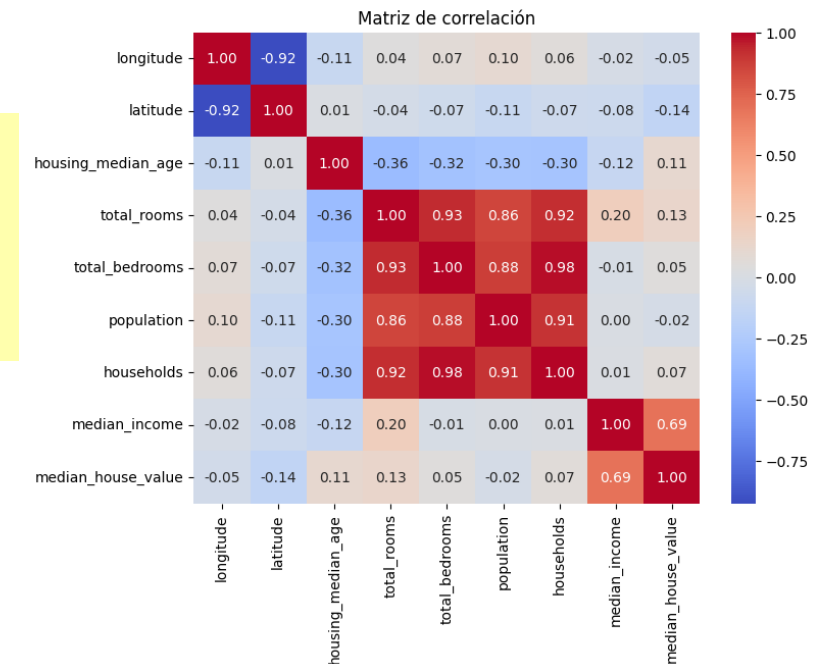
plt.imshow(corr, cmap='coolwarm', interpolation='none')
plt.colorbar()
plt.xticks(range(len(corr)), corr.columns, rotation=90)
plt.yticks(range(len(corr)), corr.columns)
plt.title('Mapa de calor (correlaciones)')
plt.show()
```



Mapa de calor - seaborn

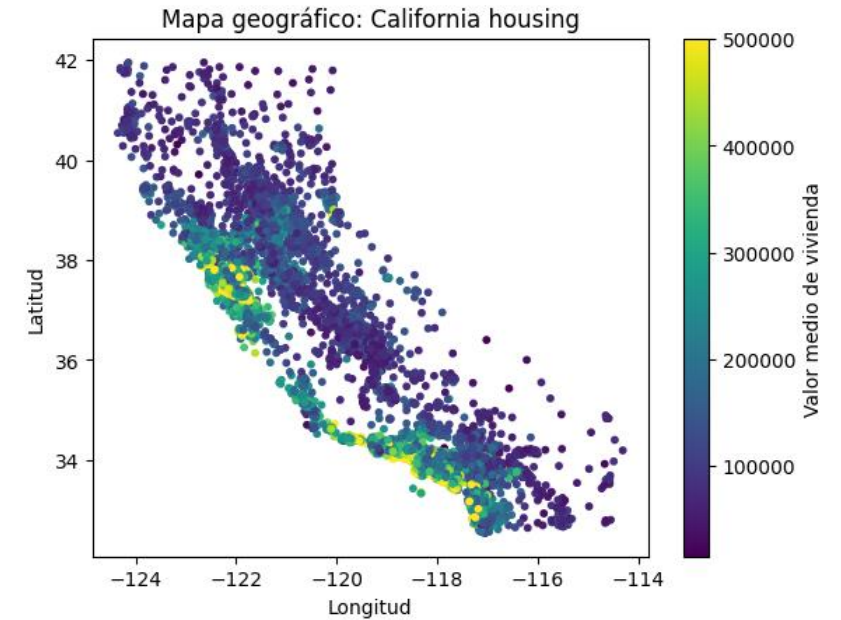
```
plt.figure(figsize=(8, 6))
corr = housing_df.select_dtypes(include='number').corr()

sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Matriz de correlación')
plt.show()
```



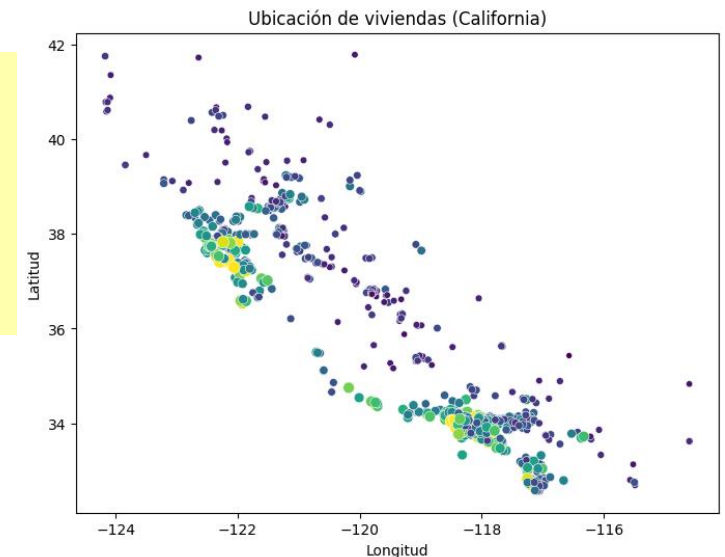
Mapa con datos geográficos - matplotlib

```
plt.scatter(housing_df['longitude'], housing_df['latitude'],  
            c=housing_df['median_house_value'], cmap='viridis', s=10)  
plt.colorbar(label='Valor medio de vivienda')  
plt.title('Mapa geográfico: California housing')  
plt.xlabel('Longitud')  
plt.ylabel('Latitud')  
plt.show()
```



Mapa con datos geográficos - seaborn

```
plt.figure(figsize=(8, 6))  
sns.scatterplot(data=housing_df.sample(1000), x='longitude', y='latitude',  
               hue='median_house_value', palette='viridis', size='median_house_value', legend=False)  
plt.title('Ubicación de viviendas (California)')  
plt.xlabel('Longitud')  
plt.ylabel('Latitud')  
plt.show()
```



3. VISUALIZACIÓN DE DATOS: MATPLOTLIB, SEABORN, PLOTLY

Gráficos combinados (*subplots*): aspectos a tener en cuenta

Se define el lienzo como una **cuadrícula de 1 x 2**:
1 fila, dos columnas

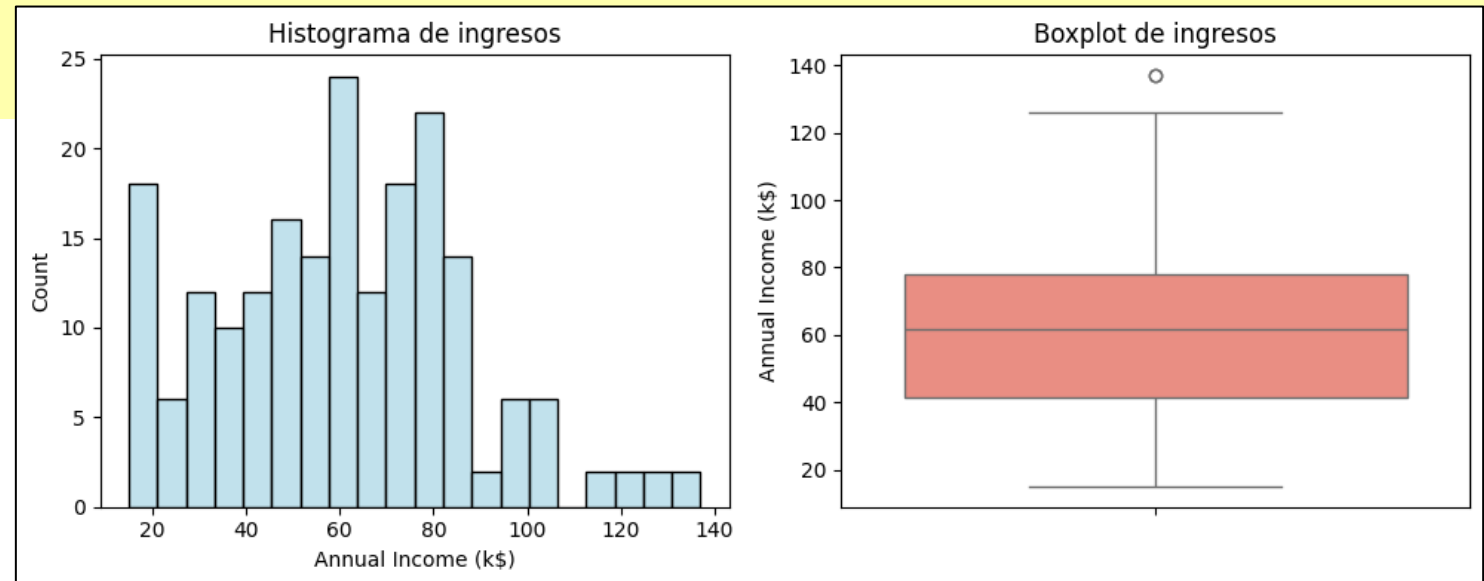
```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

```
# Histograma
sns.histplot(mall_df['Annual Income (k$)'], bins=20, ax=axes[0], color='lightblue')
axes[0].set_title('Histograma de ingresos')
```

```
# Boxplot
sns.boxplot(data=mall_df, y='Annual Income (k$)', ax=axes[1], color='salmon')
axes[1].set_title('Boxplot de ingresos')
```

```
plt.tight_layout()
plt.show()
```

Se indica en que posición del lienzo (cuadrícula) colocar cada gráfico.



Asignación doble del lienzo que contendrá múltiples gráficos (fig) y los ejes (axes)

Ayuda a evitar solapamientos

Gráficos combinados (*subplots*): aspectos a tener en cuenta

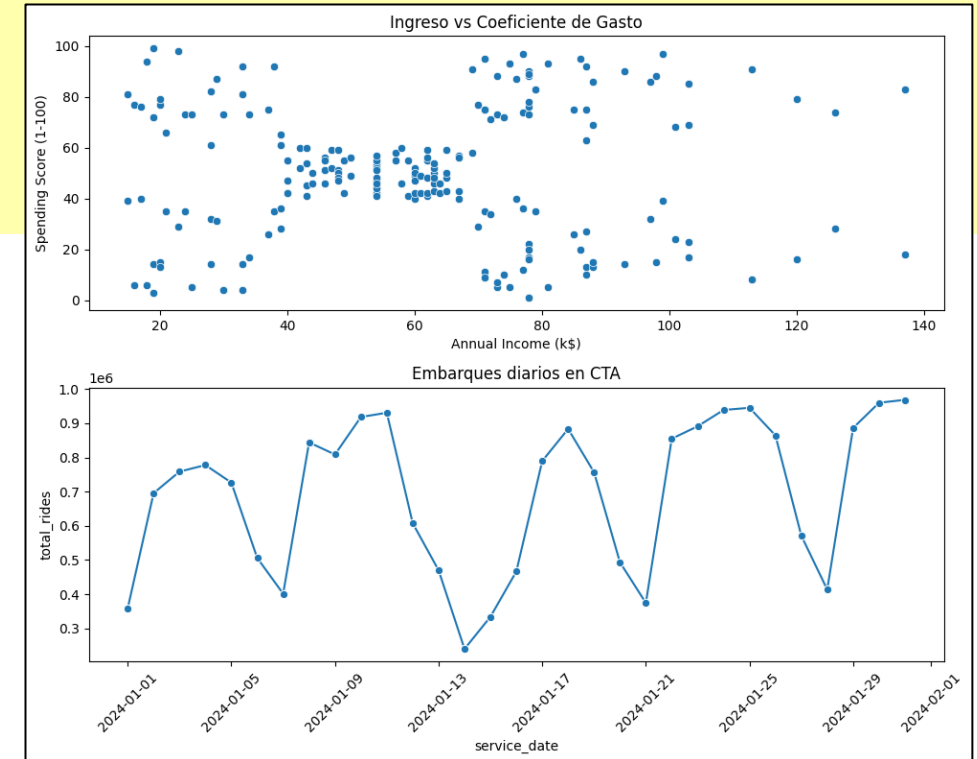
Usar "axes" como un array para especificar propiedades y elementos opcionales de cada subgráfico

```
fig, axes = plt.subplots(2, 1, figsize=(10, 8))

# Diagrama de Dispersión
sns.scatterplot(data=mall_df, x='Annual Income (k$)', y='Spending Score (1-100)', ax=axes[0])
axes[0].set_title('Ingreso vs Coeficiente de Gasto')

# Gráfico de líneas
sns.lineplot(data=cta_sample, x='service_date', y='total_rides', marker='o', ax=axes[1])
axes[1].set_title('Embarques diarios en CTA')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```



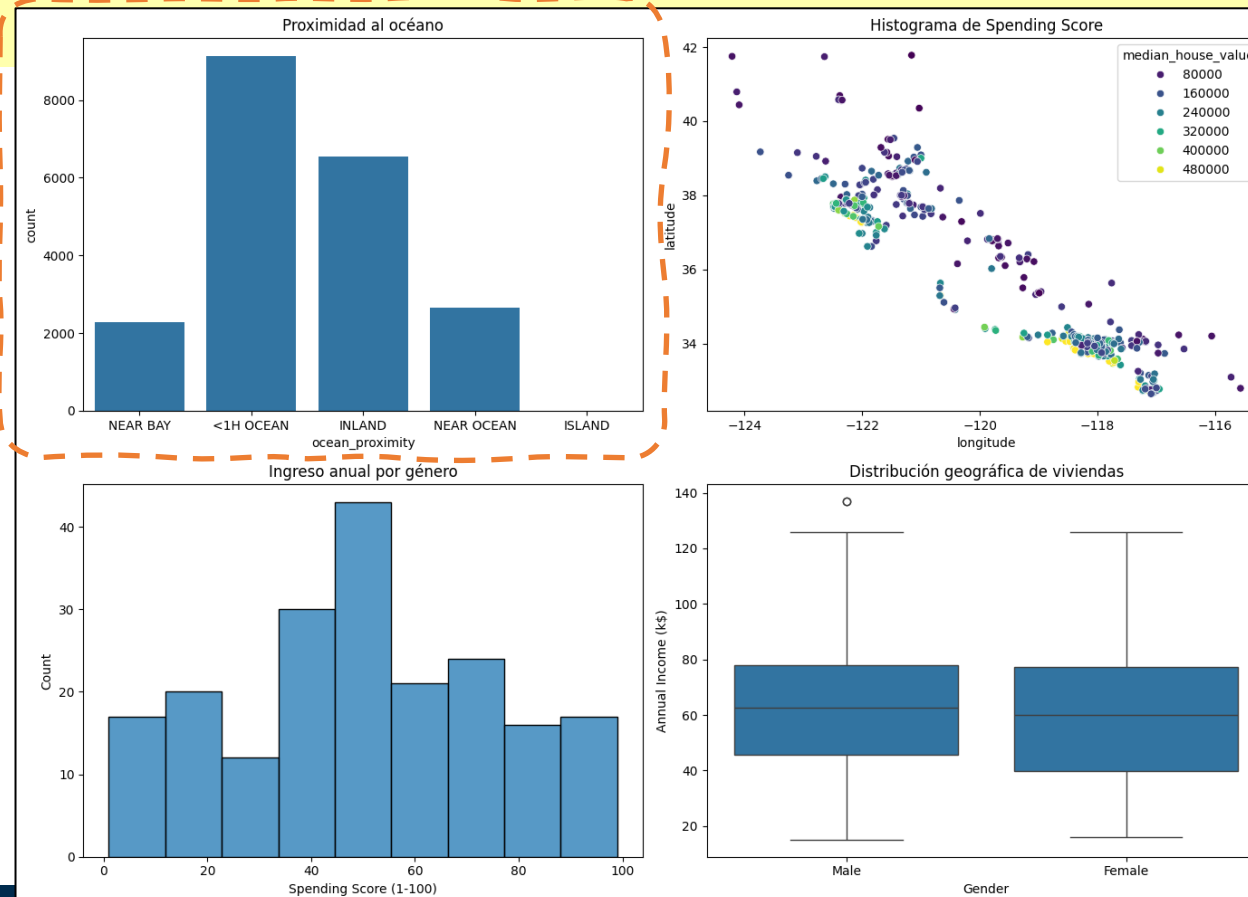
Gráficos combinados (*subplots*): aspectos a tener en cuenta

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Countplot
sns.countplot(data=housing_df, x='ocean_proximity', ax=axes[0, 0])
axes[0, 0].set_title('Proximidad al océano')

(...)
```

Si las dimensiones de la cuadrícula son ambas ≥ 2 , la variable `axes` hay que manejarla como un array bidimensional



3. VISUALIZACIÓN DE DATOS: MATPLOTLIB, SEABORN, PLOTLY

Gráficos combinados (*subplots*): aspectos a tener en cuenta

Los argumentos `sharex` y `sharey` permiten que múltiples subgráficos compartan el mismo eje x (o eje y).

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 6), sharex=True)

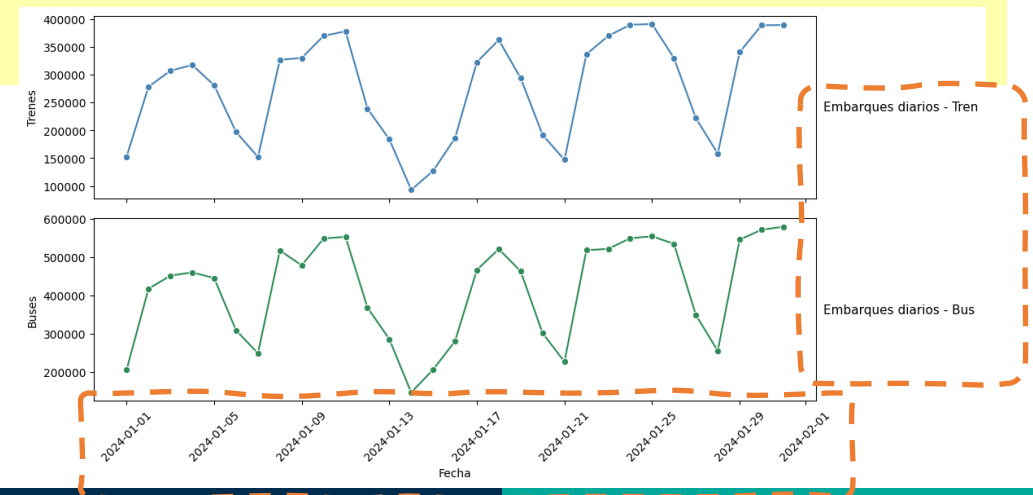
# Gráfico de líneas para tren
sns.lineplot(data=cta_sample, x='service_date', y='rail_boardings', ax=ax1, marker='o',
color='steelblue')
ax1.set_ylabel('Trenes')
ax1.text(1.01, 0.5, 'Embarques diarios - Tren', transform=ax1.transAxes,
fontSize=11, va='center', ha='left')

# Gráfico de líneas para bus
sns.lineplot(data=cta_sample, x='service_date', y='bus', ax=ax2, marker='o', color='seagreen')
ax2.set_ylabel('Buses')
ax2.set_xlabel('Fecha')
ax2.tick_params(axis='x', rotation=45)
ax2.text(1.01, 0.5, 'Embarques diarios - Bus', transform=ax2.transAxes,
fontSize=11, va='center', ha='left')
```

Truco más avanzado para colocar título de gráfico a la derecha: 1.01 y 0.5 son posiciones relativas a `ax1`

Se pueden desempaquetar los "ejes" del lienzo - objetos `Axes` - a modo de tupla

	service_date	day_type	bus	rail_boardings	total_rides
0	2010-02-01	W	1091041	648428	1739469
1	2006-07-15	A	603018	412144	1015162
2	2017-08-26	A	487453	446049	933502
3	2007-12-25	U	249699	111582	361281
4	2005-07-08	W	950222	591325	1541547



La librería **Plotly** ofrece un módulo, *plotly.express*: una API de alto nivel para crear gráficos interactivos de forma rápida y sencilla.

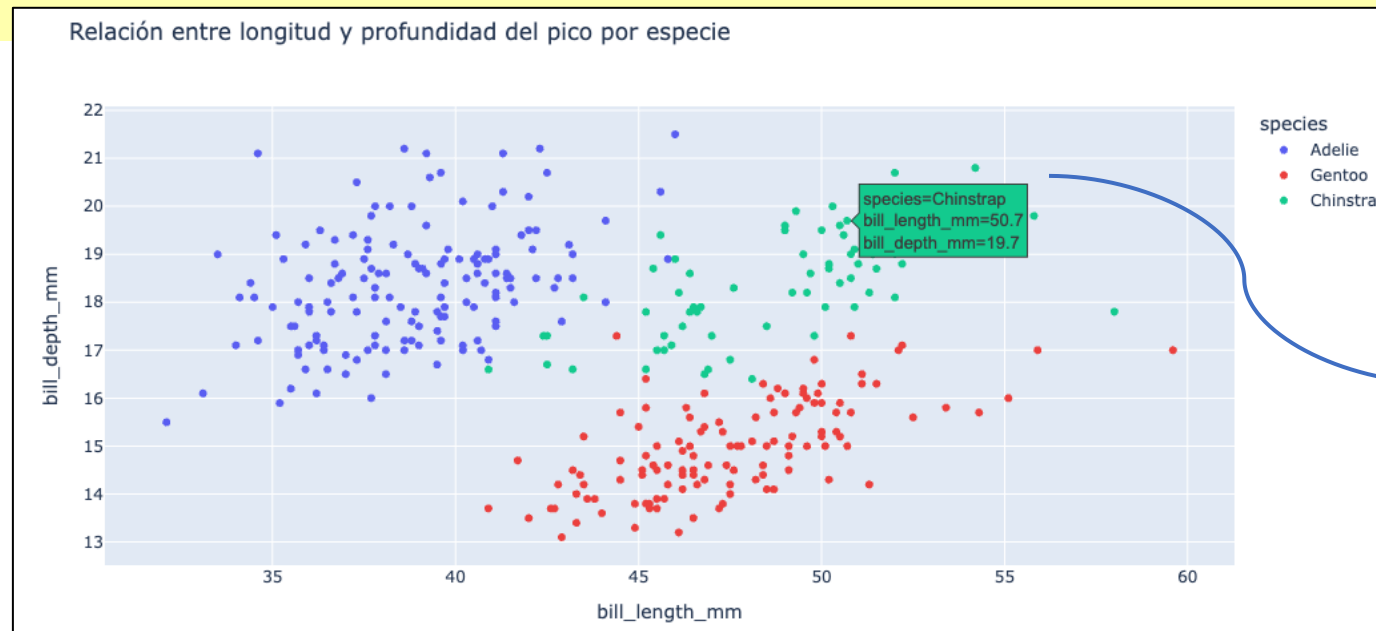
<https://plotly.com/python/plotly-express/>

```
import pandas as pd
import plotly.express as px

penguins = pd.read_csv('penguins.csv')

fig = px.scatter(penguins,
                 x='bill_length_mm',
                 y='bill_depth_mm',
                 color='species',
                 title='Relación entre longitud y profundidad del pico por especie')

fig.show()
```

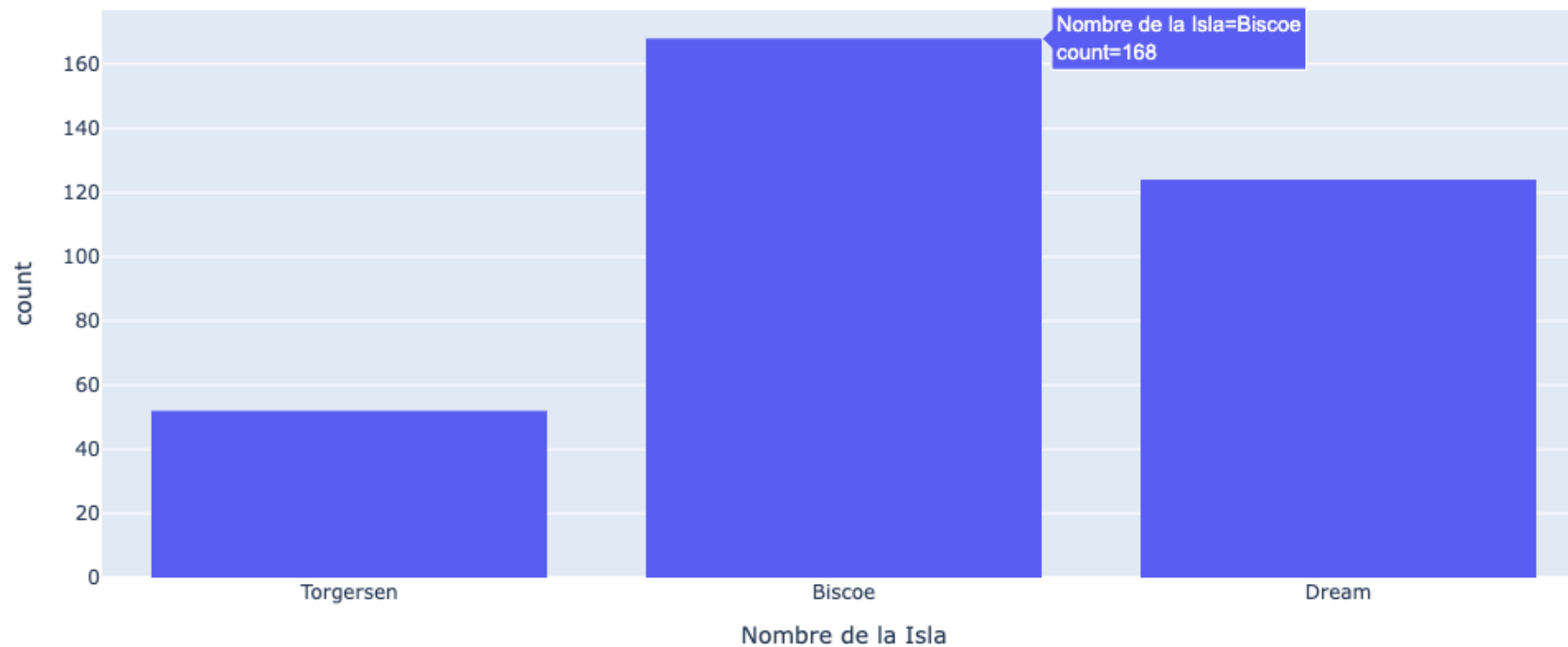


Al pasar el cursor por encima de un punto, se muestra información del mismo a través de los atributos del dataset usados en el gráfico.

```
fig = px.histogram(penguins,  
                  x='island',  
                  labels={'island': 'Nombre de la Isla'},  
                  title='Distribución de clientes por género')  
  
fig.show()
```

Personalización de
etiquetas asociadas
a atributos

Distribución de clientes por género



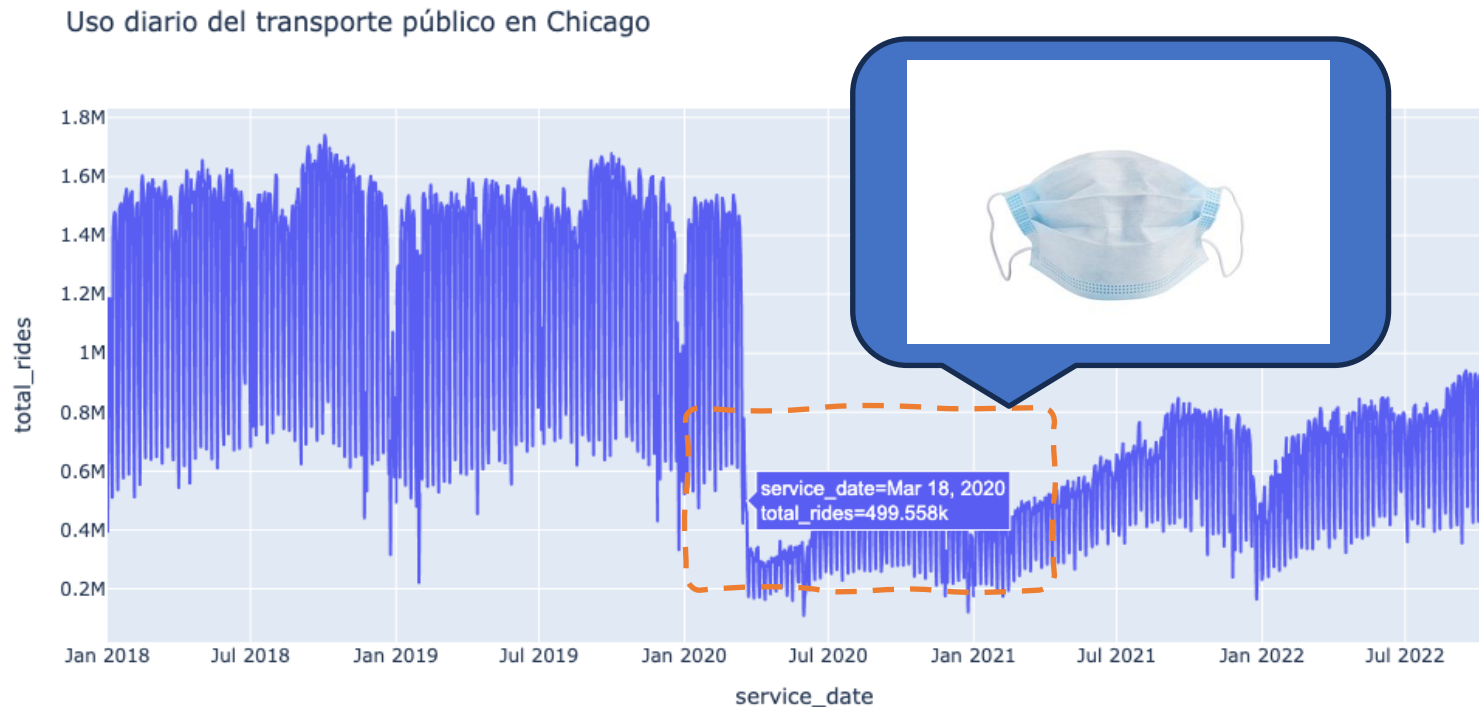
3. VISUALIZACIÓN DE DATOS: MATPLOTLIB, SEABORN, PLOTLY

```
cta = pd.read_csv('CTA_-_Ridership_-_Daily_Boarding_Totals.csv')
cta['service_date'] = pd.to_datetime(cta['service_date'])

# Gráfico de líneas del número de embarques diario en transporte público
fig = px.line(cta,
              x='service_date',
              y='total_rides',
              title='Uso diario del transporte público en Chicago')

fig.show()
```

Asegurar que la variable para el eje 'x' es de tipo fecha



3. VISUALIZACIÓN DE DATOS: MATPLOTLIB, SEABORN, PLOTLY

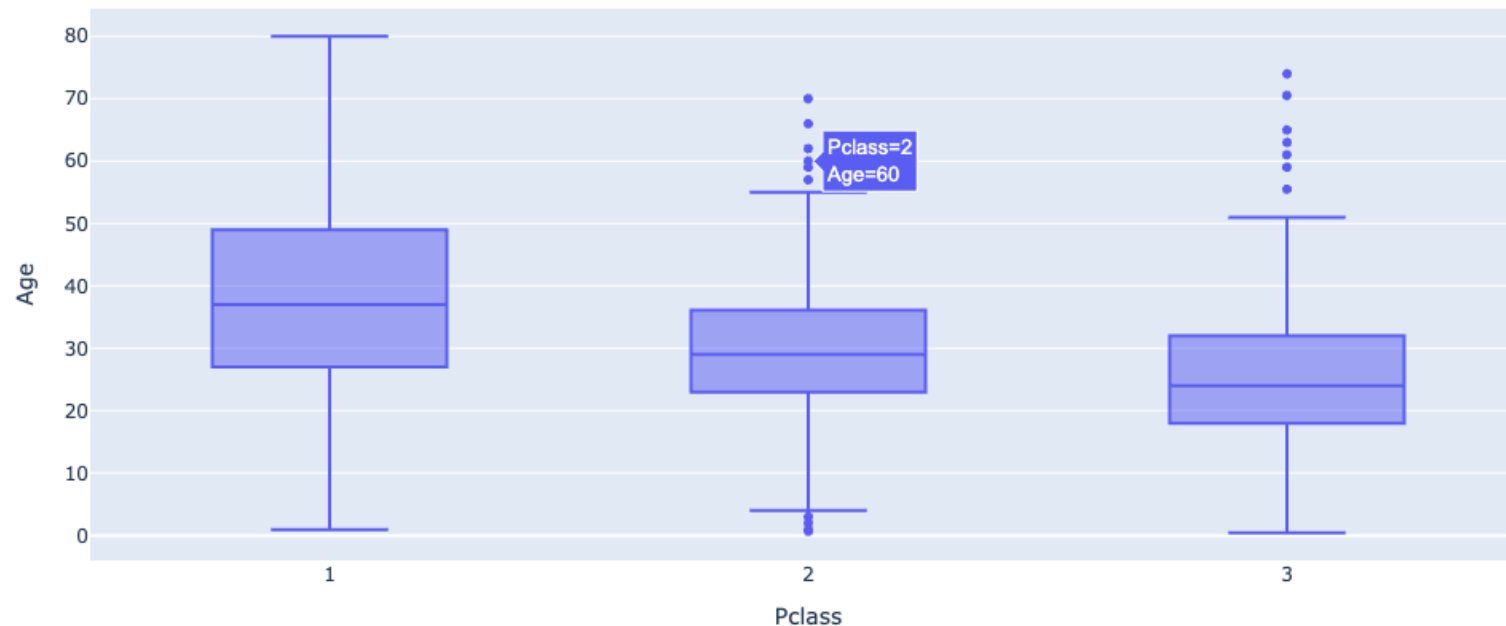
```
titanic = pd.read_csv('titanic.csv')

# Boxplot de edad separado para cada clase social
fig = px.box(titanic,
             x='Pclass',
             y='Age',
             title='Distribución de edades por clase en el Titanic')

fig.show()
```

En plotly, con el argumento x se puede especificar un atributo categórico para mostrar múltiples *boxplots*: uno por categoría

Distribución de edades por clase en el Titanic



3. VISUALIZACIÓN DE DATOS: MATPLOTLIB, SEABORN, PLOTLY

```
import pandas as pd
import plotly.express as px

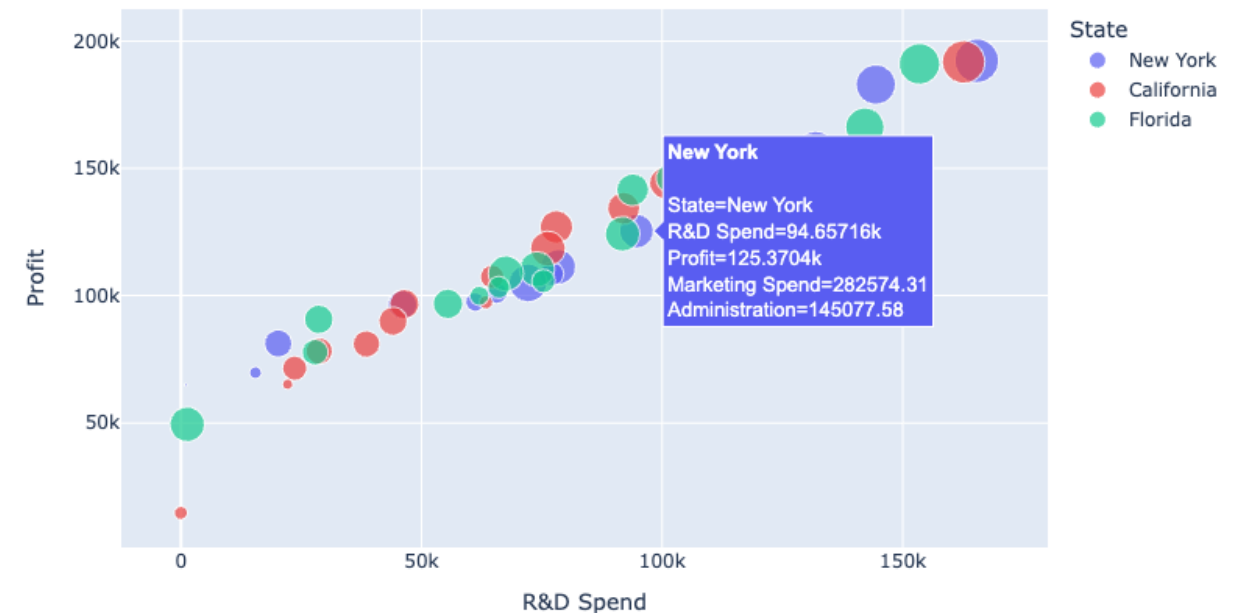
# Cargar datos
startups = pd.read_csv('50_Startups.csv')

# Crear gráfico de dispersión
fig = px.scatter(startups,
                 x='R&D Spend',
                 y='Profit',
                 color='State',
                 size='Marketing Spend',
                 hover_name='State',
                 hover_data=['Administration'],
                 log_x=False,
                 size_max=20,
                 title='Relación entre gasto en
I+D y ganancias por Estado, con tamaño
según Gasto de Marketing'
)

fig.update_layout(width=800)
fig.show()
```

Un último ejemplo, con un nivel mayor de personalización

Relación entre gasto en I+D y ganancias por Estado, con tamaño según Gasto de Market

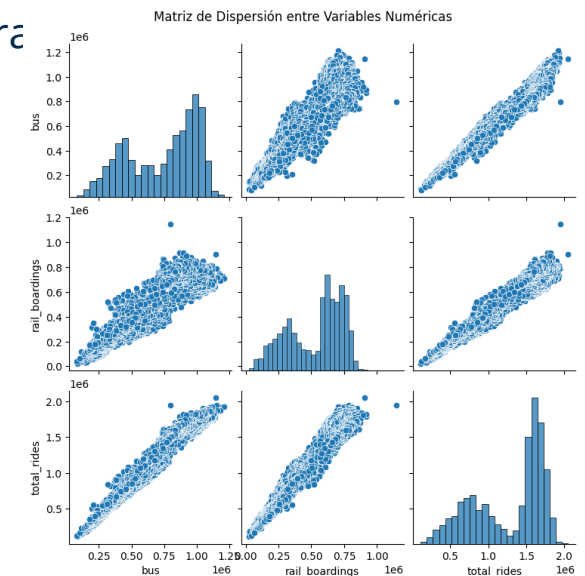


¿ Qué es el EDA?

Es un análisis preliminar de un dataset, con fines exploratorios, antes de aplicar técnicas más avanzadas como *machine learning* y uso de otros modelos de IA. Se investigan y resumen las principales características del conjunto de datos cargado en el código Python, atendiendo a:

- Inspección de los datos.
- Propiedades estadísticas básicas: media, mediana, moda, rango, desviación estándar, percentiles.
- Visualizaciones sencillas de distribuciones y variabilidad: principalmente histograma
- Distribución de los tipos de datos en los atributos.
- Detección (¡no tratamiento aún!) de valores perdidos, duplicados, outliers.

Realizar un EDA es el modo más riguroso de conocer mejor los datos con los que estás a punto de trabajar para mayores labores de análisis.



REPASO DE CONCEPTOS BÁSICOS DE ESTADÍSTICA



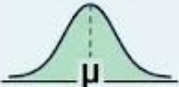
ESTADÍSTICA: REPASO RÁPIDO



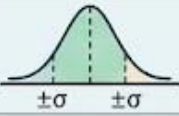
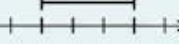
REPASO INTEGRAL DE CONCEPTOS ESTADÍSTICOS

MEDIDAS DESCRIPTIVAS

CONCEPTOS CLAVE DE ESTADÍSTICA DESCRIPTIVA

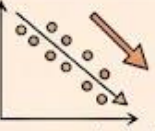
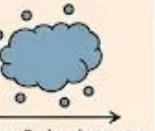
CONCEPTO	DEFINICIÓN	EJEMPLO / NOTACIÓN	VISUALIZACIÓN / FÓRMULA
Media (μ) (Poblacional)	Valor Promedio	{2, 3, 5, 7, 9} $\sum x/N$	
Mediana (Me)	Valor Central Ordenado	{2, 3, 5, 7, 9} → 5	
Moda (Mo)	Valor Más Frecuente	{2, 3, 3, 4, 5} → 3	

MEDIDAS DE DISPERSIÓN

Desviación Estándar (σ)	Cuánto se alejan los datos	$\sqrt{\frac{\sum (x - \mu)^2}{N}}$	
Rango	Diferencia Max-Min	Máx - Mín	

CORRELACIÓN Y TIPOS

COEFICIENTE DE PEARSON (r)

CONCEPTO	DESCRIPCIÓN SENCILLA	VISUALIZACIÓN / FÓRMULA
Relación Lineal	Mide relación lineal entre variables cuantitativas	Nula ($r \approx 0$)
	Positiva ($r > 0$)	
	Negativa ($r < 0$)	

**Nota: Solo detecta relaciones lineales.*

OTROS TIPOS DE CORRELACIÓN

Coeficiente de Spearman (ρ)	Relación Monótona	Basada en Rangos (datos no normales)
Coeficiente de Kendall (τ)	Mide el acuerdo entre pares de rango	$x_1 > x_2$ $y_1 > y_2$ ✓ Concordante $x_1 > x_2$ $y_1 < y_2$ ✗ Discordante

Inspección inicial de los datos y propiedades estadísticas básicas

Con este trío de sencillas funciones de pandas, podrás ...

Función	¿Qué muestra?
<code>head()</code>	Muestra las primeras filas del dataset (por defecto, 5). Útil para ver datos concretos y estructura general.
<code>info()</code>	Da información general del DataFrame: número de entradas, tipos de datos, memoria usada, y cantidad de valores no nulos.
<code>describe()</code>	Devuelve estadísticas descriptivas (media, desviación, percentiles, etc.) para columnas numéricas.

Media aritmética: dadas N instancias con valores x para el atributo, se calcula como $\mu = \frac{\sum x}{N}$

Cuartiles Q1, Q2, Q3: valores del atributo que, cuando son ordenados de forma ascendente, dejan debajo de sí el 25%, 50% y 75% de los valores más bajos, respectivamente

Desviación estándar: mide la variabilidad y dispersión del atributo.

$$\sigma = \sqrt{\frac{\sum (x - \mu)^2}{N}}$$

	service_date	bus	rail_boardings	total_rides
count	8036	8036	8036	8036
mean	2011-12-05 00:20:36.436037888	737148	530454	1267602
min	2001-01-01 00:00:00	80783	23544	110047
25%	2006-07-02 18:00:00	475264	338196	827475
50%	2011-12-01 12:00:00	819968	602002	1506496
75%	2017-05-01 06:00:00	971227	706960	1644321
max	2022-10-31 00:00:00	1211992	1146516	2049519
std	NaN	268307	208199	461997

Distribución de los atributos según tipo de dato

```
df.dtypes.value_counts()
```

int64	3
datetime64[ns]	1
object	1

"Object" suele ser indicador de un atributo identificado como categórico.

dtype: int64

Detección de valores faltantes y duplicados

```
# Valores faltantes
df.isnull().sum()

# Instancias duplicadas
print(f"Duplicados: {df.duplicated().sum()}")
```

	0
service_date	0
day_type	0
bus	0
rail_boardings	0
total_rides	0

dtype: int64

Detección manual de outliers (suele ir precedida de un boxplot)

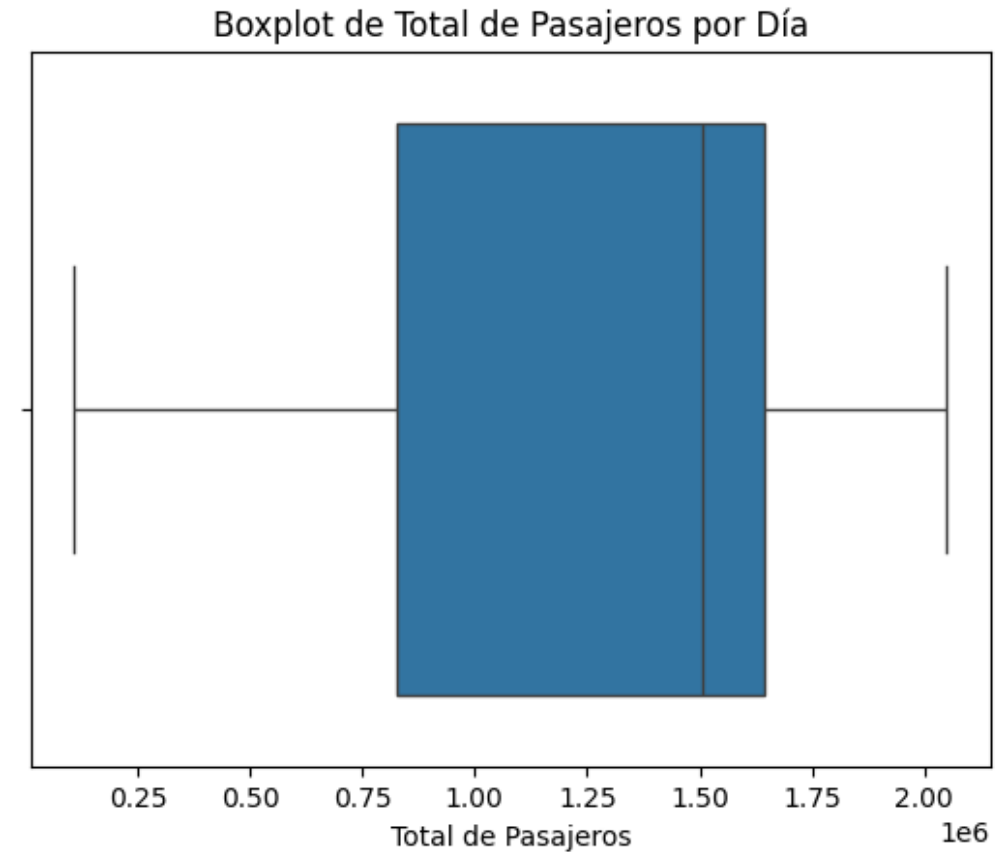
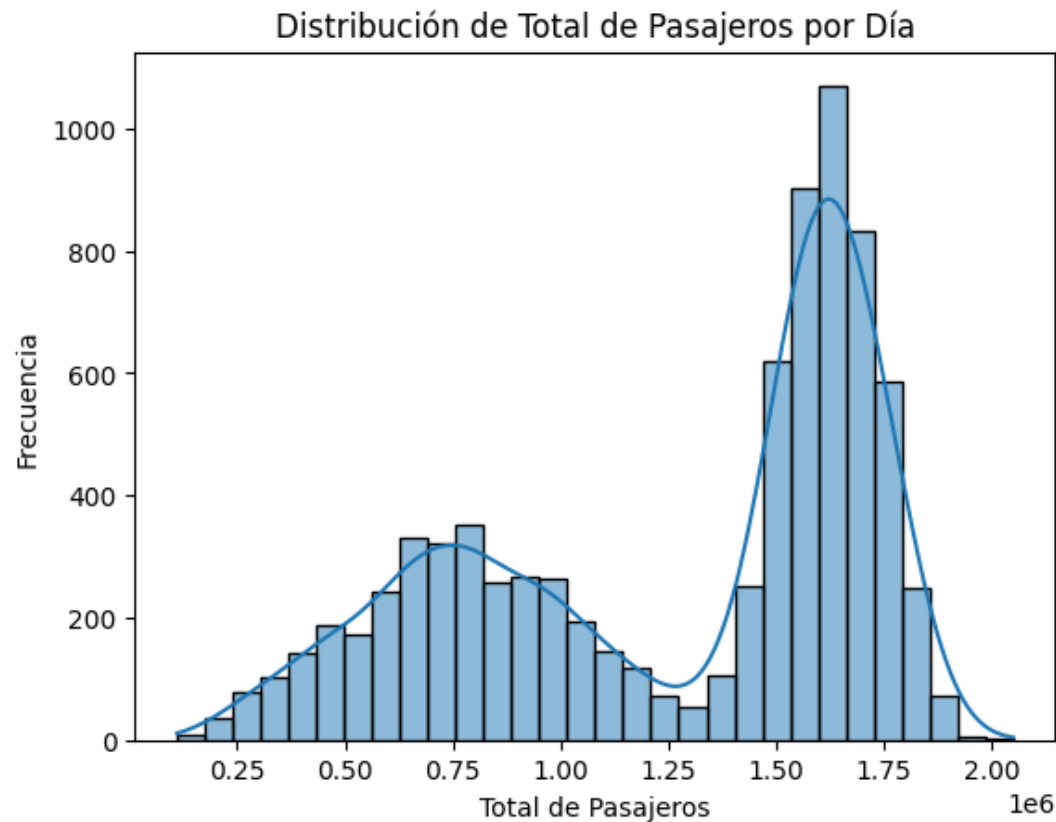
```
Q1 = df["total_rides"].quantile(0.25)
Q3 = df["total_rides"].quantile(0.75)
IQR = Q3 - Q1 # Rango intercuartílico

outliers = df[(df["total_rides"] < (Q1 - 1.5 * IQR)) | (df["total_rides"] > (Q3 + 1.5 * IQR))]
print(f"Número de outliers detectados: {outliers.shape[0]}")
```

Visualizaciones de las distribuciones de los datos

Histograma → frecuencias por categoría o intervalo numérico (*bin*)

Boxplot → Dispersión y variabilidad

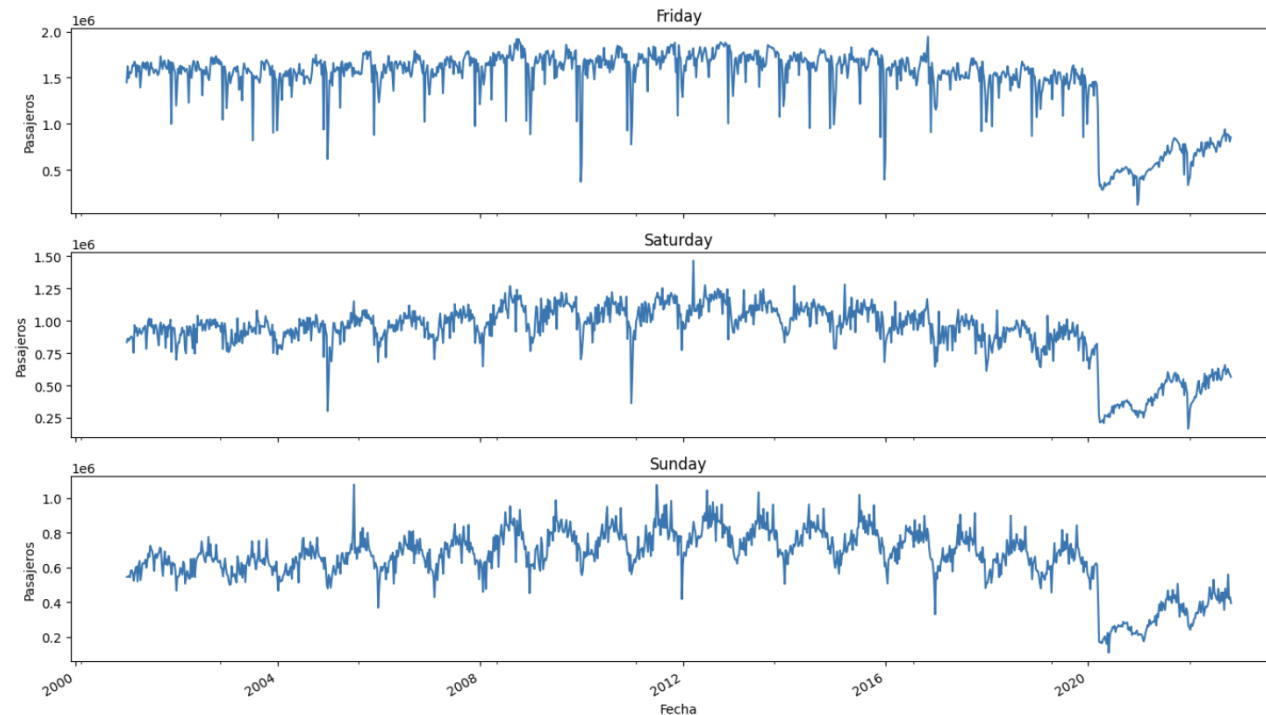


Análisis segmentados por categoría

Paso 1. Crear un nuevo atributo o elegir un atributo existente para particionar el dataset en grupos.

- Ejemplo: en el dataset “Chicago rides”, obtener el día de la semana asociado a cada fecha y clasificar los registros diarios según nuevo de atributo que contiene el día de la semana.

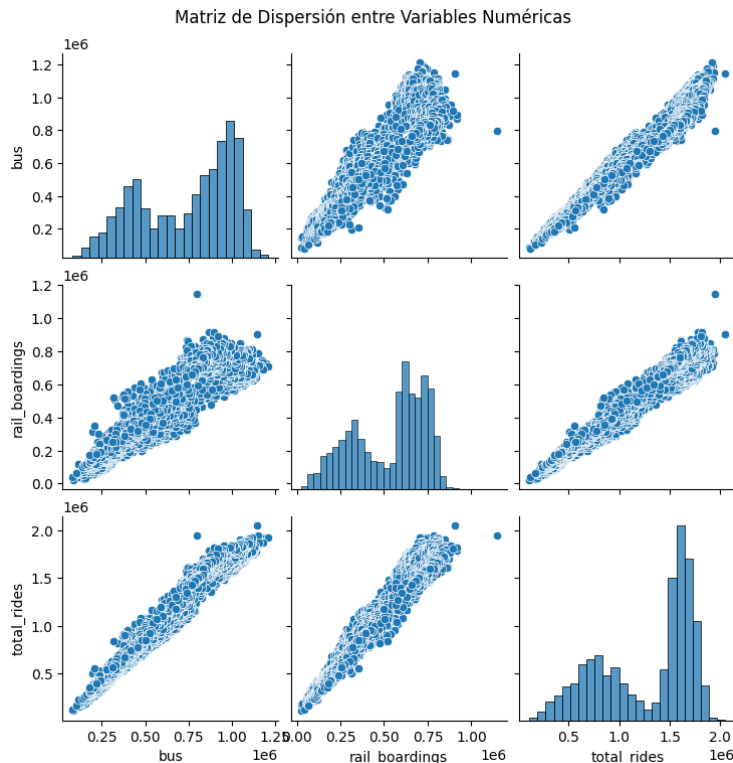
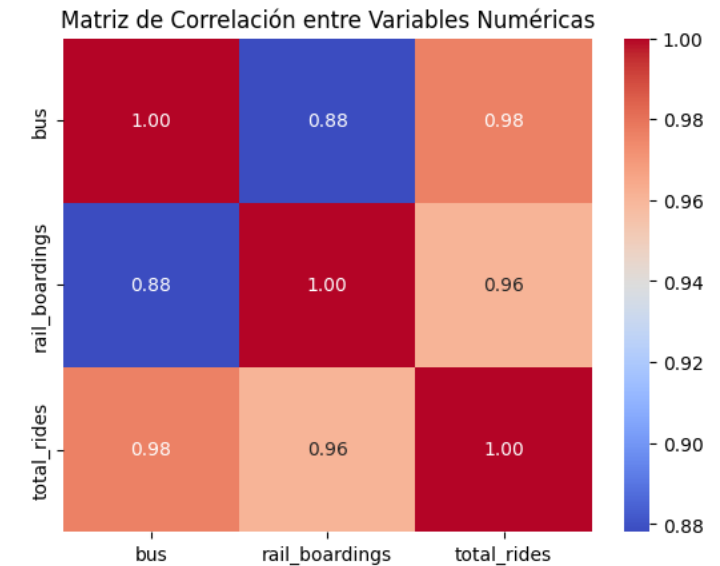
Paso 2: visualizaciones compuestas por grupo/categoría.



Análisis de correlaciones entre atributos

Matriz de correlación (ya vista en el tema anterior)

```
corr_matrix = df.corr(numeric_only=True)
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm",
            fmt=".2f")
plt.title("Matriz de Correlación entre Variables
Numéricas")
plt.show()
```



Matriz de dispersión (¡NUEVO!)

- Muestra los diagramas de dispersión (scatterplot) de cada posible par de atributos numéricos.
- En la diagonal principal, muestra el histograma del propio atributo.

```
numeric_df = df.select_dtypes(include='number')
```

```
sns.pairplot(numeric_df, diag_kind="hist")
plt.suptitle("Matriz de Dispersión entre Variables Numéricas", y=1.02)

plt.show()
```

Análisis descriptivo

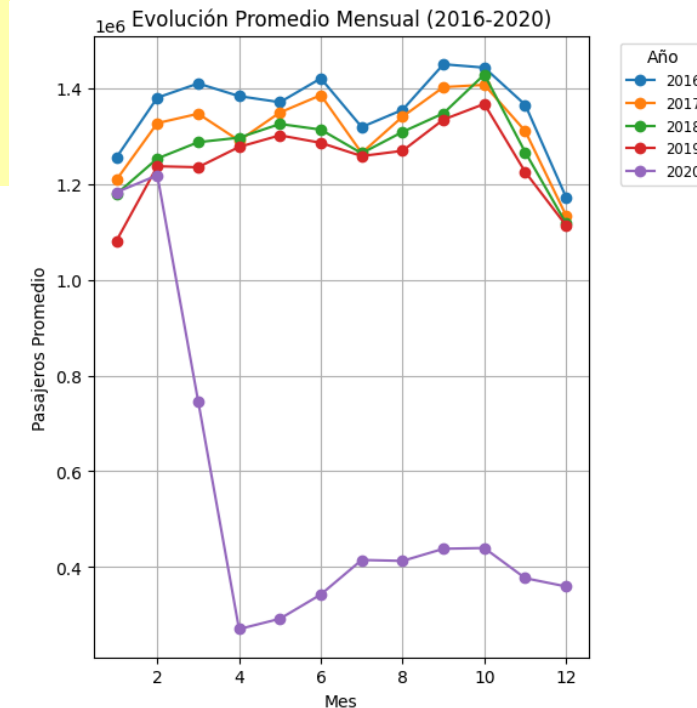
- Pone un mayor énfasis en inspeccionar, describir y resumir datos pasados y presentes.
- Enfoque y propósito muy similar al EDA, pero normalmente más profundo y metodológico, al ser no un “vistazo inicial”, sino el principal análisis a realizar en muchos casos.

Se busca identificar rigurosamente:

- Relaciones entre variables
- Tendencias
- Patrones
- Estructuras internas (p.ej. subgrupos).

Mientras que el EDA tiene un propósito más general y rutinario, el análisis descriptivo suele hacer para **responder a preguntas de Análisis específicas**. Por ejemplo:

¿Existen patrones estacionales o diferencias significativas entre años en el número promedio de pasajeros por mes?

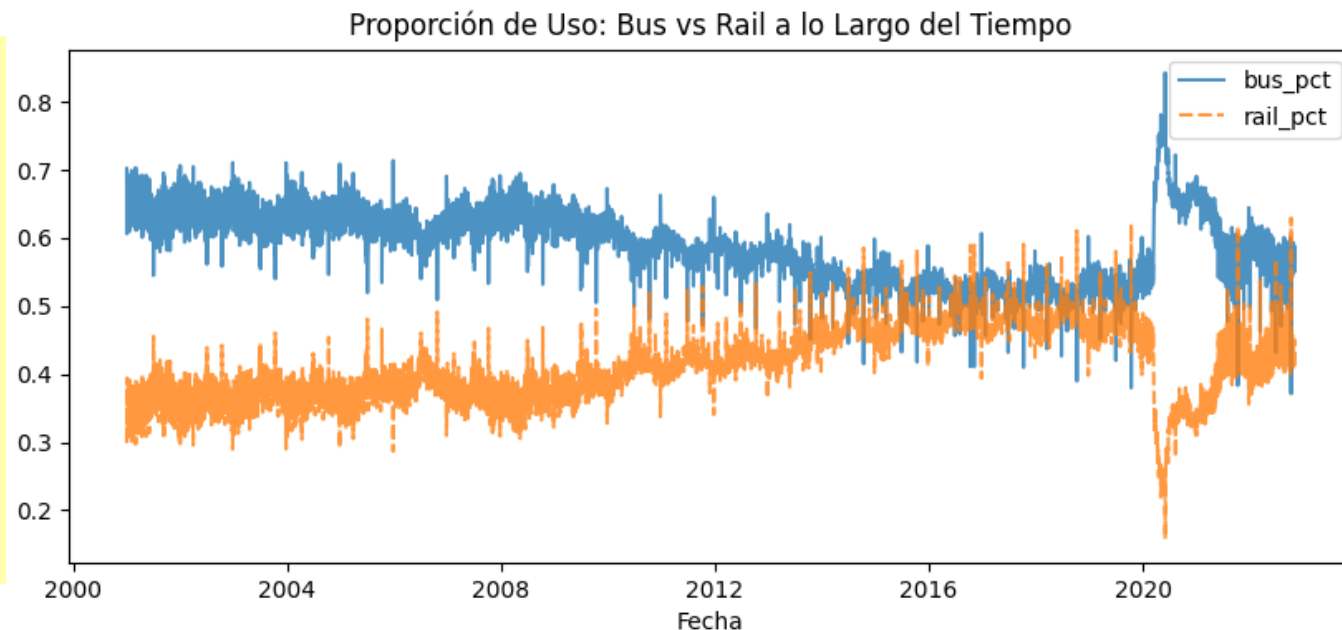


Análisis descriptivo

- **Análisis proporcional:** consiste en comparar las proporciones relativas (porcentajes) de valores asociados a dos o más atributos que son mutuamente excluyentes.
- El número de pasajeros de bus en una ciudad frente al número de pasajeros de tren, es un claro ejemplo de ello.

Se crean dos nuevos atributos que contendrán proporciones

```
transport_df = df[["bus", "rail_boardings",  
                  "total_rides"]].copy()  
transport_df["bus_pct"] = df["bus"] / df["total_rides"]  
transport_df["rail_pct"] = df["rail_boardings"] /  
                             df["total_rides"]  
  
plt.figure(figsize=(10, 4))  
sns.lineplot(data=transport_df[["bus_pct", "rail_pct"]],  
             alpha=0.8)  
plt.title("Proporción de Uso: Bus vs Rail a lo Largo del  
Tiempo")  
plt.ylabel("Proporción sobre Total")  
plt.xlabel("Fecha")  
plt.show()
```



4. ANÁLISIS EXPLORATORIO DE DATOS (EDA)

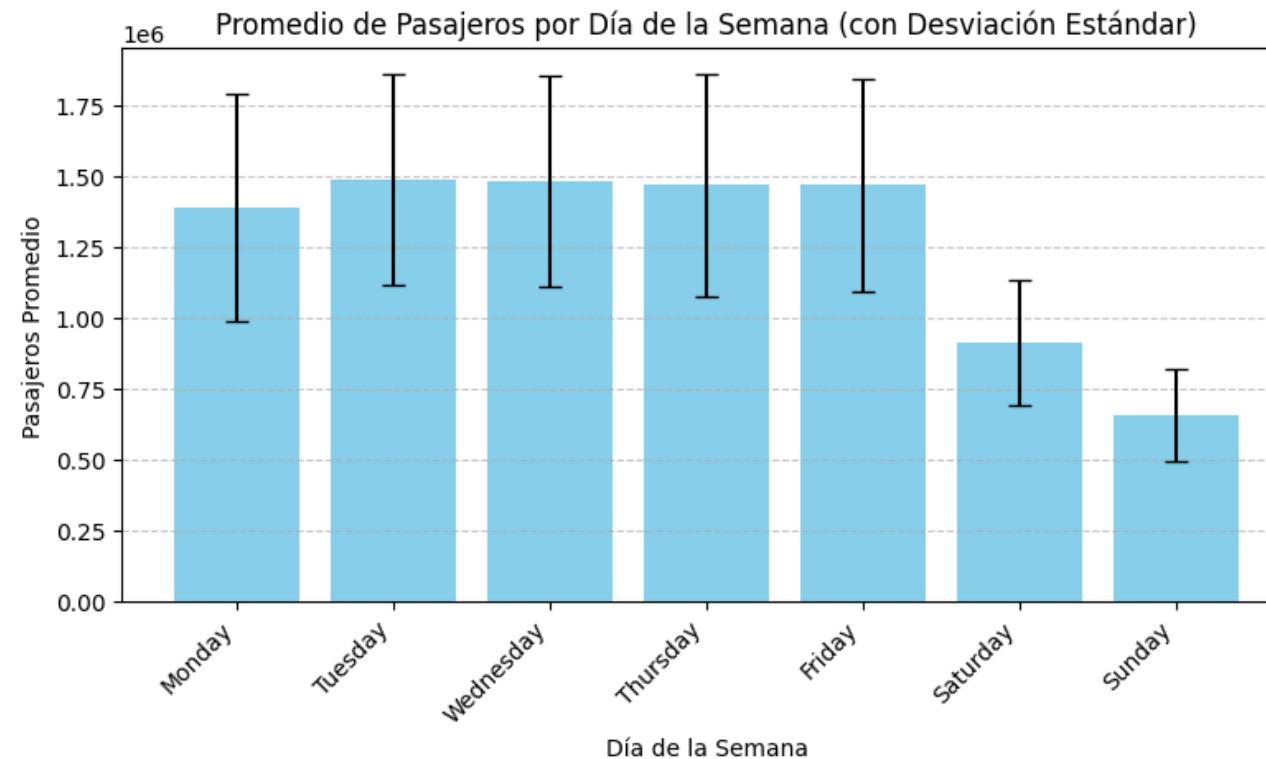
Análisis descriptivo

- Visualizar promedios junto con desviaciones estándar para varias categorías

Se crea un dataframe separado que contiene las medias y desviaciones estándar de cada uno de los 7 días de la semana

```
dow_avg = df.groupby("day_of_week")["total_rides"].agg(["mean", "std"]).reindex(dias)
...
plt.bar(dias, dow_avg["mean"], yerr=dow_avg["std"], capsize=5, color='skyblue')
...
plt.show()
```

El argumento *yerr* añade una línea de error a cada barra, en este caso con el valor de la desviación estándar a ambos lados del techo de la barra.



4. ANÁLISIS EXPLORATORIO DE DATOS (EDA)

Análisis descriptivo: Medias móviles (*rolling averages*).

- Muy útiles en series temporales, para calcular en una fecha o momento concreto, la media de los valores a lo largo de una “ventana temporal deslizante” definida, que finaliza en ese día/momento
- Ayuda a obtener una visión más suavizada de las tendencias y patrones en una serie temporal.

```
df["rolling_mean_30"] = df["total_rides"].rolling(window=30).mean()
```

```
plt.figure(figsize=(12, 5))
```

```
df["total_rides"].plot(alpha=0.4, label="Datos Diarios", linewidth=1)
```

```
df["rolling_mean_30"].plot(label="Media Móvil (30 días)", linewidth=2, color="crimson")
```

Media móvil de “total_rides” para los 30 días anteriores a la instancia (día) actual

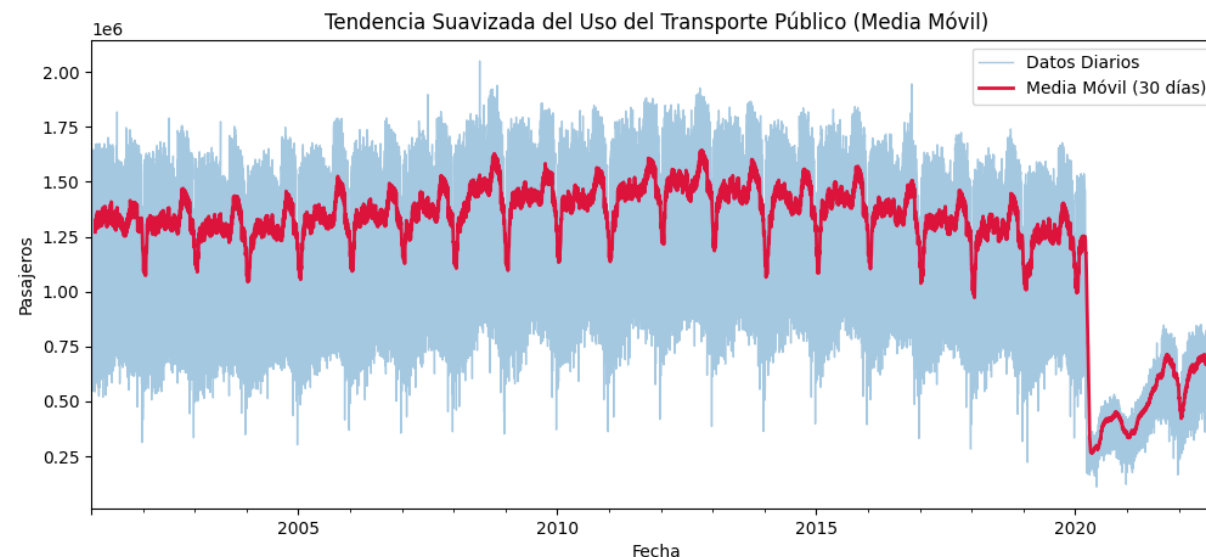


Gráfico de líneas doble: se añade al gráfico de valores aislados diarios, el de medias móviles